ROBOTICS SOFTWARE STATE OF THE PRACTICE

A STATE-OF-THE-PRACTICE STUDY OF ROBOTICS SOFTWARE FOR MOTION PLANNING AND SIMULATION

BY

ZIYANG FANG, B.Eng.

A Report

submitted to the Department of Computing and Software

and the School of Graduate Studies

of McMaster University

in partial fulfilment of the requirements

for the degree of

Master of Engineering

Master of Engineering (2026) McMaster University

(Department of Computing and Software) Hamilton, Ontario, Canada

TITLE:	A State-of-the-Practice Study of Robotics Software for Motion Planning and Simulation
AUTHOR:	Ziyang Fang B.Eng. (Mechanical Design, Manufacturing and Automation), Fuzhou University, School of Mechanical Engineering, China
SUPERVISOR:	Spencer Smith Matthew Giamou
NUMBER OF PAGES:	xxii, 156

Lay Abstract

Robots are usually tested in software simulations before they are trusted in the real world. The quality of the software that plans robot motions and simulates robot behaviour therefore matters to everyone who builds or studies robots. This report examines 28 widely used open-source packages for robot motion planning and simulation and grades each one on qualities such as how easy it is to install, how well it is documented, and how actively it is maintained, using only publicly visible evidence. The study finds that this software is generally in good shape: installation support and documentation are strong, although written design information is often missing. The report also shows that a package's popularity does not always match its measured quality.

Abstract

Software for motion planning and simulation (MPS) supports the design, testing, and deployment of robotic systems. Because experiments and applications depend on this software behaving correctly, its engineering quality matters beyond any single project. This report assesses the state of the practice for MPS software by adapting the research-software assessment methodology of Smith et al. Candidate packages were gathered from academic surveys, community-curated lists, and auxiliary LLM-assisted discovery; after consolidation of 295 unique candidate names and filtering by open-source availability, repository-based measurability, and maintenance activity, 28 packages were selected, spanning full simulators, physics and dynamics engines, motion planning libraries, and middleware. Each package was measured in May 2026 against a standard template covering installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency, together with repository metrics. A commonality analysis describes the selected packages as a software family.

The measured packages show strong practical infrastructure: all 28 provide installation instructions, installation automation, tutorials, user manuals, and API documentation, and only two installations broke during measurement. Weaknesses

concentrate in formal artifacts: only one package documents expected user characteristics, and half do not identify a coding standard. Measured quality and community popularity diverge for several packages, so popularity indicators such as GitHub stars are not reliable proxies for engineering quality. Compared with prior State-of-the-Practice studies of medical imaging and Lattice Boltzmann method software, MPS software is at least as well equipped in repository artifacts and development tooling. A developer-interview component has been designed to investigate pain points and practices; its execution is left for future work.

Acknowledgements

I would like to thank my supervisor, Dr. Spencer Smith, for his guidance and feedback throughout this work. I would also like to thank Dr. Matthew Giamou, who served as the domain expert for this study, for reviewing the candidate software list and for his advice on the scope, inclusion, and exclusion decisions.

Contents

List of Figures

List of Tables

Notation, Definitions, and Abbreviations

Notation

CR	Consistency ratio (Analytic Hierarchy Process)
CI	Consistency index (Analytic Hierarchy Process; not to be confused with CI = Continuous Integration in the Abbreviations list)
RI	Random index for a pairwise-comparison matrix of a given size (Analytic Hierarchy Process)
n	Dimension of a pairwise-comparison matrix (Analytic Hierarchy Process)
$\lambda_{\max}$	Principal eigenvalue of a pairwise-comparison matrix

Definitions

Alive package

In this report, a package is considered alive if it has received any commit or release in the 18 months preceding the measurement date.

Software family

A set of programs whose common properties are extensive enough that studying them collectively is advantageous before analysing individual members (Parnas, 1976).

State-of-the-Practice study

An assessment of the artifacts, tools, processes, and quality of existing software in a chosen domain, in contrast to a study that designs or implements new software.

Surface measure

An assessment based on shallow but systematic checks performed within a limited time budget rather than on exhaustive experimental evaluation.

Abbreviations

AHP	Analytic Hierarchy Process
AIST	National Institute of Advanced Industrial Science and Technology (Japan)

API	Application Programming Interface
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Delivery
CVC	Computer Vision Center (Barcelona)
DART	Dynamic Animation and Robotics Toolkit
DDS	Data Distribution Service
GIS	Geographic Information Systems
IIT	Istituto Italiano di Tecnologia (Italian Institute of Technology)
LBM	Lattice Boltzmann Method
LLM	Large Language Model
MI	Medical Imaging
MREB	McMaster Research Ethics Board
MJCF	MuJoCo XML configuration format
MPS	Motion Planning and Simulation
OMG	Object Management Group
OMPL	Open Motion Planning Library
OSS	Open Source Software
RL	Reinforcement Learning

ROS	Robot Operating System
SCC	Sloc, Cloc and Code (the source-code line-counting tool used for the repository measurements)
SDF	Simulation Description Format
SDK	Software Development Kit
SLAM	Simultaneous Localization and Mapping
SOFA	Simulation Open Framework Architecture
SOP	State-of-the-Practice
TRI	Toyota Research Institute
URDF	Unified Robot Description Format
YARP	Yet Another Robot Platform

Declaration of Academic Achievement

I, Ziyang Fang, declare that the research presented in this report was designed and carried out by me. This includes the candidate software discovery, the mention-count ranking, the repository-based measurements of the 28 selected packages, the commonality analysis, the comparison with other research software domains, and the writing of all chapters. Dr. Spencer Smith provided supervisory guidance on the methodology and on the structure and revision of the report. Dr. Matthew Giamou, as domain expert, reviewed the candidate software list and advised on scope, inclusion, and exclusion decisions. In the course of this work I made use of artificial-intelligence tools under my own direction. The exploratory trial reported in Appendix F is one such use; all validated measurements reported in Chapter 5 were performed and checked by me, and I take full responsibility for the content of this report.

Chapter 1

Introduction

Robotics software for motion planning and simulation (MPS) enables researchers and engineers to model robot kinematics and dynamics, simulate sensor data, plan collision-free trajectories, and test control strategies before deploying to physical hardware. An application of MPS software is shown in Figure 1.1.

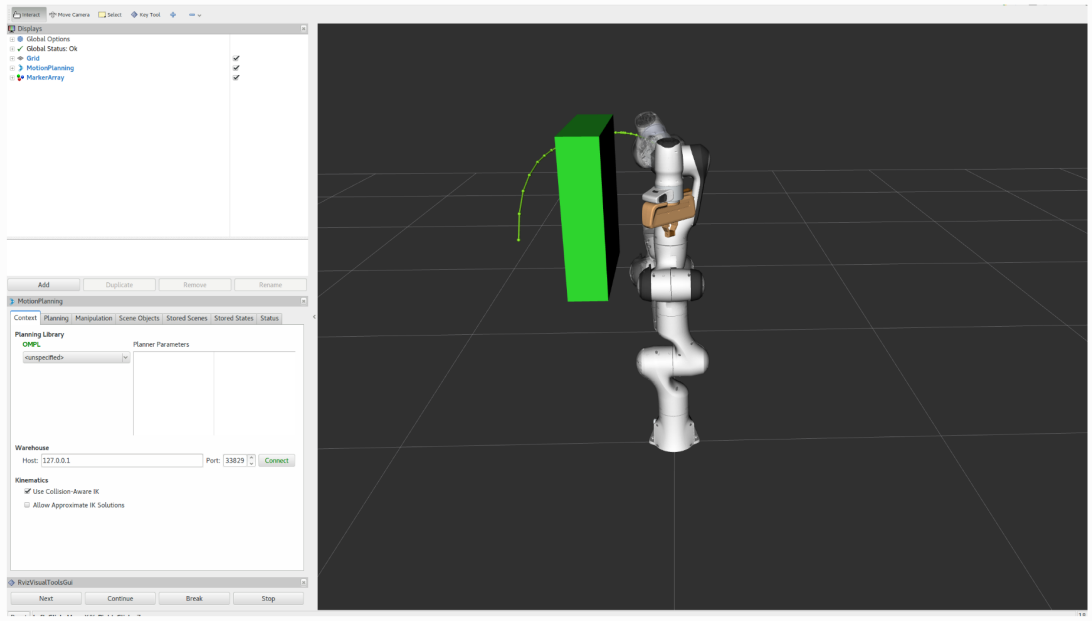

Figure 1.1: An illustrative path planning task: a manipulator computing a collision-free path around an obstacle.

In practice, this software is central to experimental reproducibility and to the transition from simulation to physical deployment, so its engineering quality has direct consequences for both research and operational outcomes. A simulator that produces incorrect trajectories, or a planner that crashes under realistic inputs, can compromise experimental results. Qualities such as correctness, reliability, and long-term sustainability are important for MPS software.

Software engineers have developed methods and tools for improving software quality: version control, automated testing, and continuous integration are standard practice in professional contexts. However, a gap exists between these practices and what researchers actually do when developing scientific software. Scientists and engineers working on research software tend to learn software engineering skills informally,

picking them up from colleagues or through trial and error rather than through formal training (Hannay et al., 2009). Hannay et al. (2009) observe that many scientists show indifference to standard software engineering concepts, and Prabhu et al. (2011) report that more than half of the scientists they surveyed did not use a debugger. Because much of the software studied in this report originates in research settings, these concerns apply directly to the MPS domain.

Previous State-of-the-Practice studies have investigated this gap across several scientific software domains, including geographic information systems (Smith et al., 2018a), mesh generation (Smith et al., 2016), seismology (Smith et al., 2018c), statistical software for psychology (Smith et al., 2018b), medical imaging software (Smith et al., 2024a), and Lattice Boltzmann method software (Smith et al., 2024b). This report adapts the same methodology (Smith et al., 2021) to MPS software. The aim is to understand how MPS software holds up against established software engineering practices and how it compares with other research software domains. The comparison identifies where MPS developers already meet those practices and where gaps remain. The report ranks the selected packages according to the defined quality attributes; these rankings describe the current state of practice in software development, rather than serving as recommendations for any particular software package.

This chapter is organized as follows. Section 1.1 states the research questions, while Section 1.2 defines the scope of the study. A high-level overview of the methodology is provided in Section 1.3. Section 1.4 describes the contributions of the report, and Section 1.5 outlines the remaining chapters.

1.1 Research Questions

The following research questions guide this report.

RQ1. What software packages are candidates for the State-of-the-Practice study of MPS software?

RQ2. What is the ranking of the candidate packages from RQ1 with respect to the software qualities of installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency?

RQ3. How do MPS projects compare to other research software domains with respect to the artifacts present in their repositories?

RQ4. How do MPS projects compare to other research software domains with respect to the use of tools?

RQ5. How do MPS projects compare to other research software domains with respect to the principles, processes, and methodologies used?

RQ6. What are the pain points for developers working on MPS software projects?

RQ7. How do the pain points for MPS developers compare to the pain points for developers in other research software domains?

RQ8. For MPS developers, what specific best practices address the pain points identified in RQ6 and the software qualities mentioned in RQ2?

RQ9. What research software development practices, beyond those already used by MPS developers, address the pain points identified in RQ6?

RQ1 covers candidate discovery; RQ2 concerns the measurement and ranking of the selected packages from public artifacts. RQ3–RQ5 compare MPS software with prior State-of-the-Practice studies of other research software domains. RQ6–RQ9 concern the developer perspective on pain points, best practices, and recommendations for future practice.

1.2 Scope

This report covers MPS software: robotics packages for motion planning, dynamics, physics, simulation, and supporting middleware. To be included, a package must expose sufficient public evidence, such as a source repository, documentation, issue tracker, releases, or project metadata, to support consistent repository-based measurement. Tools whose core components are proprietary or otherwise closed therefore fall outside the measured set even if they are widely used in robotics practice. Chapter 3 states the full inclusion and exclusion criteria and gives the rationale for individual tools that were considered and removed.

The selected packages play different roles within shared MPS workflows; Chapter 4 explains those roles before the measurement results are interpreted.

1.3 Overview of the Methodology

This report adapts the State-of-the-Practice methodology (Smith et al., 2021). The scope and exclusions were reviewed with a domain expert to reduce the risk of misrepresenting the MPS domain. At a high level, the methodology consists of the following steps:

1. Define the MPS software domain and selection criteria.
2. Identify candidate packages from academic sources, community-curated lists, and auxiliary LLM-assisted discovery.
3. Filter the candidate list using domain relevance, public measurability, and repository-based evidence.
4. Measure and rank the selected packages relative to one another using repository artifacts, documentation, releases, issue trackers, and project metadata.
5. Analyze the selected packages as a software family through commonality analysis.
6. Compare the MPS findings with prior State-of-the-Practice studies of other, more recently studied research software domains.
7. Prepare the developer-interview component needed to supplement the artifact-based results with evidence about developer pain points and best practices.

1.4 Contributions

This report makes the following contributions:

1. A scoped State-of-the-Practice study of MPS software, applying the methodology of Smith et al. (2021) to a new domain.
2. A transparent candidate discovery and selection process that combines academic sources, community-curated lists, and auxiliary LLM-assisted discovery, with explicit discussion of the role and limitations of each source type.

3. A public-artifact measurement dataset for 28 selected software packages, covering repository evidence, documentation, releases, issue trackers, testing and continuous-integration evidence, and project metadata, organized across the nine quality categories defined in the measurement template of Smith et al. (2021), whose generic State-of-the-Practice methodology is referred to as the Domain X methodology throughout this report.

4. A commonality analysis identifying shared capabilities, variability points, and parameters of variation across the selected packages.

5. A comparison of MPS software with findings from prior State-of-the-Practice studies in other research software domains (Smith et al., 2024b,a, 2018c).

These contributions are descriptive and methodological. The report characterizes existing software by measuring it against defined quality attributes. The resulting rankings describe the current state of practice in software development; they do not prescribe any particular software package.

1.5 Organization

The remainder of this report is organized as follows. The research questions are not confined to a single chapter; they are answered throughout the body, and each chapter notes the research questions it addresses to maintain traceability back to Section 1.1.

- **Chapter 2** introduces the robotics software domain and the motion planning and simulation context, reviews prior State-of-the-Practice studies, defines the software quality attributes, provides background on commonality analysis and

the Analytic Hierarchy Process, and discusses the limitations of open-source repository-based measurement.

- **Chapter 3** describes the methodology: domain selection, candidate discovery from multiple source types, filtering criteria, the measurement procedure using the Domain X template (Smith et al., 2021), the commonality analysis procedure, and the developer interview component with its adapted Domain X interview protocol, recruitment, and analysis plan.

- **Chapter 4** presents the 28 selected packages as a software family and reports the commonality analysis: shared capabilities, variability points, and parameters of variation.

- **Chapter 5** ranks the selected packages on the nine Domain X quality categories, reports the measurement results and repository metrics, and compares the quality-based ranking with community-facing indicators of visibility and adoption.

- **Chapter 6** compares MPS software with prior State-of-the-Practice studies of other scientific software domains on three dimensions: repository artifacts, tool usage, and development principles, processes, and methodologies.

- **Chapter 7** describes the planned developer-interview component for MPS software and identifies the interview evidence still needed to answer RQ6–RQ9: the evidence base, recurring pain points, practices used to address them, quality-related findings, and cross-domain comparison.

- **Chapter 8** gives recommendations for future MPS software practice, based

on the public-artifact measurements, commonality analysis, and cross-domain comparison.

- **Chapter 9** discusses threats to validity, including limitations related to candidate selection, public-artifact measurement, source bias, and the interpretation of quality scores.
- **Chapter 10** summarizes the study, answers the research questions, states the key findings, and identifies directions for future work.

Chapter 2

Background

This chapter sets out the concepts the rest of the report relies on. Section 2.1 introduces the MPS domain. Section 2.2 defines the software categories relevant to this report. Section 2.3 reviews the State-of-the-Practice methodology and its prior applications. Section 2.4 defines the nine software quality attributes measured in this report. Section 2.5 introduces commonality analysis as a method for describing software families. Section 2.6 introduces the Analytic Hierarchy Process used to support ranking. Section 2.7 discusses the role and limitations of open-source repository-based measurement.

2.1 Robotics Software for Motion Planning and Simulation

MPS software supports the design, testing, evaluation, and deployment of robotic systems. Motion planning software generates feasible robot motions under constraints

such as geometry, kinematics, dynamics, collisions, and task requirements (LaValle, 2006). Simulation software provides virtual environments for representing robots, sensors, actuators, controllers, and physical interactions before, or alongside, work with physical hardware (Collins et al., 2021).

The MPS domain in this report is broader than a single class of software. It covers full simulation environments such as Gazebo, Webots, CoppeliaSim, and CARLA, physics and dynamics engines such as Bullet, MuJoCo, ODE, and DART, motion planning libraries such as OMPL, manipulation frameworks such as MoveIt!, and middleware such as ROS 2 and YARP. These tools are not interchangeable but often appear in the same workflow: a simulator may call a physics engine for contact and dynamics, a planning stack may sit on top of a collision-checking library and communicate through middleware, and middleware may wire planning, control, perception, and simulation together. Figure 2.1 sketches these relationships among the four software roles.

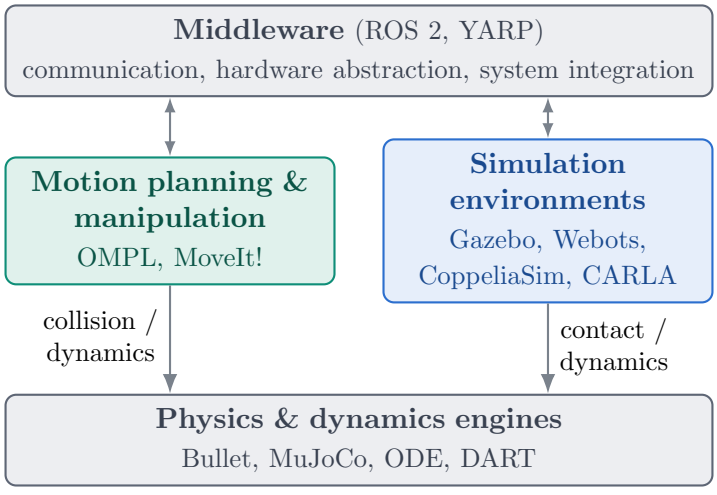

Figure 2.1: Representative MPS software roles. Box contents are example packages from the selected set.

This breadth creates a scope challenge for a State-of-the-Practice study. A narrow definition restricted to full simulators would make comparison simpler, but it would exclude software that practitioners routinely use in motion planning and simulation workflows. A broader definition better reflects the practical ecosystem, provided the report is explicit about the different roles that software packages play within the MPS domain.

2.2 Software Categories

Assessing software packages requires understanding the licensing model under which each package is distributed. In particular, the availability of source code determines whether repository-based measurement is possible. Following the terminology used in prior State-of-the-Practice studies (Smith et al., 2024a,b), this report distinguishes three common software categories:

Open Source Software (OSS): For OSS, the source code is openly accessible. Users have the right to study, change, and distribute it under a license granted by the copyright holder. For many OSS projects, the development process relies on collaboration among contributors worldwide. Accessible source code exposes the underlying logic of software functions, the development practices used to produce them, and the flaws and potential risks in the final product. OSS is therefore suitable for the kind of repository-based analysis performed here.

Freeware: Freeware is software that can be used free of charge. Unlike OSS, the authors of freeware do not permit access to or modification of the source code. To many end users, the distinction between freeware and OSS may not matter.

However, for researchers seeking insight into software development processes, inaccessible source code is a barrier to measurement.

Commercial Software: Commercial software is developed and sold by companies as a product. Typically, commercial software requires payment to access all features and does not include access to the source code. Some commercial software is available free of charge, but the lack of source access prevents the kind of repository-based assessment used in this report.

This report assesses only software that fits under the Open Source Software category. This restriction is a methodological necessity, not a judgement about the quality or importance of freeware or commercial tools.

2.3 State-of-the-Practice Studies

A State-of-the-Practice study examines existing tools, artifacts, processes, and evidence in a chosen domain. The goal is not to build new software but to learn from what already exists, by collecting and organizing public evidence in a systematic way. Claims about development activity, documentation, issue management, installation behaviour, or maintainability must be backed by observable project artifacts. When evidence is missing or ambiguous, the gap is recorded rather than filled with assumptions.

The methodology used in this report is based on "Methodology for Assessing the State of the Practice for Domain X" by Smith et al. (2021). It is generic enough to apply to any research software domain, and its authors estimate a full assessment at roughly 173 person-hours, close to their stated goal of one person-month (160 hours),

so that a small team can complete it. The prescribed process moves from domain selection to measurement and analysis. It starts with selecting a suitable domain and engaging a domain expert. The candidate software list is then constructed and filtered, followed by a domain analysis. Source code, documentation, and repository metrics are collected for the selected packages, and a measurement template covering nine quality categories is applied to each. Developers are surveyed through semi-structured interviews, and the Analytic Hierarchy Process (AHP) is used to support cross-quality ranking. The procedure ends with answers to a set of research questions about the domain.

This methodology builds on prior State-of-the-Practice assessments conducted across several research software domains: Geographic Information Systems (Smith et al., 2018a), Mesh Generators (Smith et al., 2016), Seismology software (Smith et al., 2018c), and Statistical software for psychology (Smith et al., 2018b). More recently, the methodology was refined and applied to Medical Imaging (MI) software (Smith et al., 2024a) and Lattice Boltzmann Method (LBM) software (Smith et al., 2024b; Michalski, 2021). These studies are the primary structural and stylistic references for this report.

The MI and LBM reports share a common structure. They open with motivation and background, describe the methodology, present the measurement results by quality category, compare those results with community rankings, report developer interviews and pain points, discuss the research questions, and close with recommendations and a conclusion. This report follows that same outline and adapts it to the MPS domain.

Key features of the Domain X methodology that distinguish it from ad hoc software comparisons include: (i) the involvement of a domain expert to vet the software list, domain analysis, and ranking (Chapter 3 describes how the domain expert was involved in this study); (ii) a transparent candidate discovery process using multiple source types; (iii) a standardized measurement template that applies the same set of fields (approximately 108, counting substantive questions and supplementary entries together) to every package; (iv) the use of repository-based metrics alongside qualitative assessments; and (v) a commonality analysis that treats the selected packages as members of a software family rather than as unrelated products.

Practitioner-facing research software checklists provide complementary guidance on themes such as source control, licensing, documentation, testing, automation, distribution, and governance (Acton, 2026). These themes overlap with several Domain X quality categories, but this report uses the Domain X template as its measurement instrument.

2.4 Software Quality Attributes

This report treats software quality operationally: a package exhibits quality to the extent that observable attributes support dependable installation, use, modification, reuse, and understanding. Following common practice in software engineering, quality is decomposed into distinct attributes, often called "ilities" (Smith et al., 2021). This report assesses nine quality attributes drawn from the measurement template in Appendix B of Smith et al. (2021); this template is reproduced, as adapted for this study, in Appendix C of this report. Several of the qualities use the word "surface" to indicate that the assessment is based on shallow but systematic checks performed

within a limited time budget, not on exhaustive experimental evaluation (Smith et al., 2024a).

Installability: The effort required for the installation and uninstallation of software in a specified environment. Measured through the presence and quality of installation instructions, the number of steps required, the availability of automation, the handling of dependencies, and the presence of validation procedures.

Correctness and Verifiability: A program is correct if it matches its specification, which may be stated explicitly or implicitly. The related quality of verifiability is the ease with which the software components or the integrated product can be checked to demonstrate correctness. Measured through the presence of requirements or theory documentation, techniques used to build confidence in correctness (such as automated testing, assertions, or continuous integration), and the availability and quality of tutorials with expected outputs.

Surface Reliability: The probability of failure-free operation of a computer program in a specified environment for a specified time. Surface reliability is assessed through whether the software breaks during installation or initial tutorial use, whether descriptive error messages are displayed, and whether recovery is possible.

Surface Robustness: Software possesses robustness if it behaves reasonably when it encounters circumstances not anticipated in the requirements specification, and when users violate the assumptions in its requirements specification. Surface robustness is assessed through whether the software handles unexpected or malformed input gracefully, including edge cases such as modified line endings

in text input files.

Surface Usability: The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. Surface usability is assessed through the presence of getting-started tutorials, user manuals, documented user characteristics, and the availability and variety of user support channels.

Maintainability: The effort with which a software system or component can be modified to correct faults, improve performance or other attributes, or satisfy new requirements. Assessed through version information, contribution and review guidance, available non-code artifacts, issue tracking practices, the percentage of identified issues that are closed, the percentage of code that consists of comments, and the version control system in use.

Reusability: The extent to which a software component can be used with or without adaptation in a problem solution other than the one for which it was originally developed. Assessed through the number of code files and the presence and quality of API documentation.

Surface Understandability: The capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use. Surface understandability is assessed through a review of randomly selected source files, examining indentation consistency, coding standards, identifier quality, use of named constants, comment clarity, algorithm references, parameter ordering, and modularization.

Visibility and Transparency: The extent to which all steps of a software development process and the current status of that process are conveyed clearly to external observers. Assessed through whether the development process is defined and documented, whether the development environment is documented, and whether release notes are published.

These nine qualities organize public evidence into comparable categories. They are not complete evaluations of each package. Scores may vary with the use case, platform, user background, or measurement date. Later chapters therefore report both the measurements and their limitations.

2.5 Commonality Analysis

A domain analysis consists of systematically identifying and documenting the commonalities, variabilities, and parameters of variation of a software family (Weiss, 1998). A software family is defined as "a set of programs whose common properties are so extensive that it is advantageous to study the common properties of the programs before analyzing individual members" (Parnas, 1976). Weiss (1998) defines commonality analysis as an approach to defining a family by identifying three kinds of information:

Commonalities: Goals, theories, models, definitions, and assumptions that are shared across family members. For example, all robotics simulators share the goal of representing robot kinematics and dynamics in a virtual environment.

Variabilities: Goals, theories, models, definitions, and assumptions that differ between family members. For example, simulators may differ in the physics engines they use, the sensor models they support, or the robot platforms they can represent.

Parameters of Variation: The possible values for each variability, along with their binding time. Binding time records when a particular variation is fixed: at specification time (when the software is designed), at build time (when the software is compiled), at configuration time (when the software is set up for use), or at run time (when the software is executing).

Commonality analysis was added to the Domain X methodology after early applications revealed that software packages within a domain can vary considerably, even when they appear to belong to the same family (Smith et al., 2024b). Without it, comparisons risk treating fundamentally different types of software as if they were interchangeable. The analysis records where the packages do similar things and where their roles diverge, so a low score on, for example, installability means different things for a full simulator and a focused dynamics library.

In this report, commonality analysis carries the role-sensitive interpretation: it identifies shared concerns across full simulators, physics engines, planning libraries, and middleware while documenting the dimensions on which packages differ.

2.6 Analytic Hierarchy Process

The Domain X methodology uses the Analytic Hierarchy Process (AHP) to combine the nine quality impression scores into an overall ranking that reflects the relative

importance of each quality (Saaty, 1980). AHP is a multi-criteria decision-making technique developed by Saaty in the 1970s. It structures a decision problem as a hierarchy with a goal at the top (here, overall software fitness), criteria in the middle (the nine measured qualities), and alternatives at the bottom (the 28 measured packages). The relative importance of the criteria is elicited through pairwise comparisons rather than asking the decision-maker to assign absolute weights, which is generally easier and more reliable in practice.

For each pair of criteria, the decision-maker expresses a judgement on Saaty's fundamental scale. The odd values carry the named intensities: 1 means equally important, 3 moderately more important, 5 strongly more important, 7 very strongly more important, and 9 extremely more important. The even values (2, 4, 6, 8) cover intermediate cases between the named intensities. The judgements populate a square reciprocal matrix whose principal right eigenvector, normalized to sum to one, yields the criterion weights. Combining these criterion weights with the per-criterion alternative scores gives an overall ranking of the alternatives.

AHP includes a built-in consistency check. The consistency index $CI = (\lambda_{\max} - n)/(n-1)$, where $\lambda_{\max}$ is the principal eigenvalue of the comparison matrix and n is the matrix dimension, is divided by a random-index value RI tabulated for matrices of each size to produce the consistency ratio $CR = CI/RI$. Saaty's convention is that $CR \leq 0.10$ indicates acceptable consistency; larger values suggest that the pairwise judgements should be revisited. This check is one reason AHP is preferred in Domain X over a simple unweighted mean of the per-quality scores: the mean treats all qualities as equally important regardless of context, and offers no mechanism for the assessor to make that weighting explicit or to test whether the weighting is

internally coherent.

In a State-of-the-Practice study, AHP is an analysis aid for cross-quality comparison under a stated weighting, not a universal recommendation of any particular software package. The same per-package measurements can yield different rankings under different reasonable weightings, which is why prior studies in the Domain X line report a sensitivity analysis alongside the headline ranking. Chapter 3 describes how AHP would be applied to the measurements in this report. Because a defensible criterion-weighting elicitation has not been completed at the time of writing, this report does not present an elicited-weight AHP ranking. Chapter 5 instead presents the per-package quality-mean ranking and an explicit comparison with community indicators, together with an illustrative AHP computation under equal category weights that demonstrates the weighting step without standing in for an elicited result.

2.7 Open Source and Repository-Based Measurement

The measurement process in this report depends on evidence from public artifacts. Public repositories can reveal commit history, release activity, issue tracking, documentation, source code organization, testing practices, dependencies, and community interaction. Public documentation and project websites can add evidence about installation procedures, usage patterns, supported platforms, and design goals. This dependence on public evidence makes open-source availability a prerequisite for inclusion in the selected set.

Repository-based measurement has inherent limitations. Repository metadata

represents a snapshot in time: last commit dates, issue counts, release information, and pull request activity can change after measurement. Some projects distribute their development activity across multiple repositories or use external issue trackers, making it difficult to capture a complete picture from any single repository. The interpretation of repository metrics requires care: a high number of open issues may indicate an active user community rather than poor maintenance, and a low commit frequency may reflect project maturity rather than abandonment.

The measurement record therefore documents measurement dates, repository choices, and interpretation rules. The dependence on public evidence also shapes which packages enter the selected set. Proprietary tools and tools with closed core components may be important in robotics practice, but they cannot be measured with the same procedure as open-source software. Chapter 3 discusses the specific inclusion and exclusion decisions that follow from this constraint.

Chapter 3

Methodology

This chapter describes the methodology used to conduct the State-of-the-Practice study of MPS software and addresses RQ1 (candidate identification) by documenting how the candidate set was assembled. The methodology follows Smith et al. (2021) and adapts it to the MPS domain. Section 3.1 explains how the domain was chosen, and Section 3.2 describes the role of the domain expert. Section 3.3 covers the three-source candidate discovery process; Section 3.4 introduces the mention-count ranking that orders those candidates; and Section 3.5 sets out the filtering criteria applied to the ranked list. The resulting selected set of 28 packages appears in Section 3.6. Section 3.7 then walks through the measurement procedure using the Domain X template, Section 3.8 outlines the commonality analysis, and Section 3.9 describes the developer interview component.

3.1 Domain Selection

The first step of the Domain X methodology is to identify a suitable research software domain. To be applicable for the methodology, the chosen domain must meet four criteria (Smith et al., 2021): (i) it must have a well-defined and stable theoretical underpinning, ideally supported by standard textbooks; (ii) there must be a community of researchers and practitioners studying the domain; (iii) the software packages in the domain must include open-source options; and (iv) a preliminary search should suggest that there are close to 30 candidate software packages that qualify as research software.

The selected domain is *MPS (motion planning and simulation)*. Motion planning and simulation are established subfields of robotics with well-defined theoretical foundations (LaValle, 2006; Choset et al., 2005). There is an active international research community, regular conferences (such as ICRA, IROS, and RSS), and a substantial body of published work. A preliminary search confirmed the existence of numerous open-source software packages spanning simulation environments, physics engines, dynamics libraries, motion planning libraries, and robotics middleware.

The domain boundary follows the MPS scope defined in Chapter 2, Section 2.1, rather than a simulator-only scope.

Earlier project discussions considered a narrower domain focused on inverse kinematics software. After consultation with the supervisor and domain expert, the scope was broadened to the MPS domain, which provides a richer set of candidate packages and better reflects the interconnected nature of robotics software ecosystems.

3.2 Interaction with the Domain Expert

The Domain X methodology emphasizes that a domain expert is an essential member of the assessment team. Pitfalls exist if non-experts attempt to produce an authoritative list of software or definitively rank packages without domain knowledge. Non-experts must rely on information available online, which has two known drawbacks: (i) online resources may contain false or inaccurate information; and (ii) online resources may omit relevant information that is so ingrained with experts that nobody thinks to record it explicitly (Smith et al., 2021).

Domain experts may be recruited from academia or industry. The only requirements are knowledge of the domain and a willingness to be engaged in the assessment process. The domain expert does not need to be a software developer, but they should be a user of domain software. Given that domain experts are likely to be busy people, the methodology is designed to minimize the burden on their time (Smith et al., 2024a).

For this report, the domain expert is Dr. Matthew Giamou of the Department of Computing and Software at McMaster University, whose research includes robotics, motion planning, and state estimation. Dr. Giamou reviewed the candidate software list and provided feedback on scope and coverage. He confirmed that the list was reasonable and that most of the shortlisted tools were familiar to him. He noted that several packages (ROS 2, MoveIt!, and OMPL) are not full simulators but can fit within a coherent narrative if their roles are explained carefully. He also confirmed that excluding MATLAB/Simulink, Unity, NVIDIA Isaac Sim, and RaiSim on methodological grounds (closed source or non-OSS licensing for core components,

as detailed in Section 3.5.1) was acceptable and would not misrepresent the MPS domain. MRDS and USARSim, which are also excluded, were filtered on independent activity grounds (Section 3.5.3) rather than as a separate expert decision.

3.3 Candidate Software Discovery

Candidate software packages were identified from three types of sources: academic survey papers, community-curated software lists, and LLM-generated software lists. Using multiple source types reduces dependence on any single search method and provides broader coverage of the MPS domain. The collected names were consolidated to merge equivalent entries (for example, "V-REP" and "CoppeliaSim" refer to the same package), and the consolidated list was used as input to the mention-count ranking described in Section 3.4.

3.3.1 Academic Sources

Academic sources identify software packages that appear in research discussions of robotics simulators, simulation tools, and related challenges. These sources included survey papers and comparative studies, such as Harris and Conrad (2011), Collins et al. (2021), Körber et al. (2021), and Farley et al. (2022), that discuss packages such as Gazebo, Webots, V-REP/CoppeliaSim, MuJoCo, and PyBullet in the context of robotics research. The full list of academic sources used for mention counting appears in Appendix B. Academic sources connect software packages to published research contexts and provide a research-facing view of the MPS domain.

Academic sources have known limitations as the sole basis for candidate discovery.

Survey papers tend to emphasize packages relevant to a specific research question, robot type, or time period; some active tools may not appear in any of the surveys consulted, while older tools sometimes remain visible because they were prominent when a survey was written. For that reason, academic sources were combined with community-curated and LLM-assisted sources instead of being used on their own.

3.3.2 Community-Curated Sources

Community-curated sources broaden the view of robotics software practice beyond what appears in academic surveys. The source catalogue included GitHub Topic pages, "awesome" lists (curated collections such as `awesome-robotics-libraries`, `awesome-robotic-tooling`, and `awesome-motion-planning`), best-of lists such as `best-of-robot-simulators`, and other community-maintained aggregations. Sources covered topics including motion planning, robotics simulation, robotics libraries, ROS 2 ecosystem tools, multibody dynamics simulation, and general robotic tooling. The GitHub Topic pages were manually inspected as dynamic community aggregations, while the static curated-list pages were preserved as copied raw web material where available. Appendix B lists the community-curated sources individually.

These sources collect software that practitioners and community contributors consider relevant. They can reveal projects that are underrepresented in academic surveys and can show relationships between simulators, planning libraries, middleware, physics engines, and tools. Community-curated sources also vary in maintenance quality, inclusion criteria, and update frequency. They were therefore used as discovery aids rather than as indicators of software quality.

3.3.3 LLM-Assisted Sources

LLM-generated software lists were used as an auxiliary source for candidate discovery. This modifies the original Domain X methodology, which prescribes search engine queries and domain expert consultation as the primary discovery methods. The modification is recorded explicitly so that the methodology remains transparent and reproducible in principle.

The reason for including LLM-assisted discovery is practical. Large language models surface software names that appear across a wide range of public information, and they act as a broad aggregator over that data. LLM output can also be incomplete, outdated, or incorrect, so names obtained from LLM-generated lists were consolidated, filtered, and verified against public evidence before any claims in this report were based on them. In this report, LLM-assisted sources contributed to candidate discovery but did not determine the selected set, and they were never treated as authoritative.

This approach was discussed with the supervisor. The use of LLMs is technically a modification of the original methodology, but it is defensible for two reasons: (i) LLMs were used together with academic and community-curated sources, not as the sole discovery method; and (ii) an LLM, in this context, is a large-scale aggregator over existing public information, analogous in effect to a broad search over indexed web content.

3.4 Mention Count Ranking

After consolidating equivalent names across the collected sources, each unique software package was counted once per source to produce a mention count. A package that appeared in five academic papers and three community-curated lists would receive a mention count of eight. The consolidated ranking was recorded during the discovery phase and is a transparent input to the filtering process.

Consolidation produced 295 unique candidate names. The top entries in the consolidated ranking include Gazebo (22 mentions), Webots (16), Bullet/PyBullet (16), CoppeliaSim/V-REP (13), MuJoCo (12), ODE (10), ROS 2 (10), and OpenRAVE (9). These counts help organize the candidate list and make the selection process auditable. Appendix A records every candidate with at least four mentions, together with its mention count and filtering outcome, and Appendix B lists the counted sources. The full ranked list of all 295 names, including packages that did not survive the filtering step described in Section 3.5, is preserved in the supplementary project materials that accompany this report (a consolidated ranking spreadsheet and a companion source catalogue).

Mention count is a source-appearance signal and must be interpreted accordingly. It does not measure software quality, user adoption, research impact, maintainability, or the overall suitability of any particular software package. A package may appear frequently because it is historically established, because it is broadly useful across application areas, or simply because the sources collected happened to surface it. A more specialized or newer package may be under-represented even when it is technically strong. The mention count is one input to candidate filtering, not the final ranking of the software.

3.5 Filtering Criteria

The ranked candidate list was filtered to identify packages suitable for repository-based State-of-the-Practice measurement. In the Domain X methodology, the initial filters are scope, usage, and age, applied in priority order until the list reaches a manageable size of approximately 30 packages (Smith et al., 2021). In this report, these filters are implemented through three criteria: open-source availability, repository-based measurability, and activity and maintenance status. Both the initial and filtered lists are preserved for traceability.

3.5.1 Open Source Availability

Open-source availability was required because the study depends on public software artifacts for measurement. The Domain X candidate criteria require that source code be viewable and express a preference for GitHub-style repositories from which empirical measures can be collected (Smith et al., 2021). Source code, repository history, issue trackers, documentation, and release information provide the evidence base for the measurements reported in Chapter 5.

This criterion led to the exclusion of several prominent robotics tools whose core components are not open source:

- **MATLAB and Simulink** are major platforms in both robotics research and industry, widely used for algorithm development, control design, and simulation. However, MATLAB is proprietary software. Its source code is not publicly available, and its development process is not accessible through public repositories. These properties prevent the kind of repository-based measurement that this

report depends on.

- **Unity** is a widely used game engine with robotics simulation capabilities, but its core simulation engine is proprietary.
- **NVIDIA Isaac Sim** builds on NVIDIA Omniverse and provides advanced robotics simulation features, but its core platform components are not open source.
- **RaiSim** is a high-performance physics simulator for robotics, but its source code is not publicly available under an open-source license.

The exclusions above are methodological constraints, not judgements about the quality or importance of the excluded tools. Each of these platforms plays a significant role in robotics research and industry, and their exclusion limits the coverage of the study. Chapter 9 discusses this limitation further.

3.5.2 Repository-Based Measurability

Repository-based measurability requires that a package have sufficient public evidence to support the measurement process described in Section 3.7. Relevant evidence includes source repositories, commit history, issue data, documentation, release information, build or installation instructions, test artifacts, and project metadata. Packages without sufficient public evidence cannot be measured consistently and were excluded.

This criterion also governs how multi-repository projects are handled. Some robotics software systems are distributed across multiple repositories or GitHub organizations. In such cases, the repositories most representative of the core software

were selected for measurement, and this choice was documented as part of the measurement record. The same principle applies to projects that have changed names, migrated repositories, or spawned successor projects.

3.5.3 Activity and Maintenance Status

Activity and maintenance status were considered because the study focuses on contemporary software practice. In the Domain X methodology, age is one of the initial filtering criteria, measured by the last date when a change was made, with exceptions for older packages that remain highly recommended and currently in use (Smith et al., 2021). Tools that are no longer maintained provide weaker evidence about current development practices.

The primary activity indicator used in this report is the Domain X alive/dead rule: a package is considered alive if it has received commits or version releases within the last 18 months from the measurement date. The following packages were excluded on maintenance grounds:

- **MRDS** (Microsoft Robotics Developer Studio) has not received updates since 2012 and is no longer supported by Microsoft.
- **USARSim** (Unified System for Automation and Robot Simulation) is no longer actively maintained and does not represent current practice.

3.5.4 Scope and Coverage Filters

Two algorithm-level reference implementations were excluded to keep the selected packages methodologically coherent rather than for quality reasons. **GPMP2** (Gaussian Process Motion Planner 2) and **CHOMP** (Covariant Hamiltonian Optimization for Motion Planning) are optimization-based motion planning algorithms with public reference implementations. They are relevant to motion planning research but are narrower in scope than the general-purpose simulators, physics engines, planning frameworks, and middleware tools that dominate the candidate list. Including them alongside Gazebo, MoveIt!, or ROS 2 would force a comparison between full software stacks and single-algorithm libraries, which would distort the commonality analysis and the cross-package measurements. They were therefore excluded to maintain methodological consistency, not because of any quality concern.

Filtering decisions, including the alive/dead status used in Section 3.5.3, are based on a measurement-time snapshot of repository activity, last commit dates, and release status. These signals can change after measurement, so the measurement records in Chapter 5 include the measurement date.

3.6 Final Software Set

After ranking by mention count and applying the filtering criteria described above, the selected set consists of 28 software packages. Appendix A records the disposition of the leading candidates, and Appendix D details the excluded tools. Figure 3.1 summarizes the full selection pipeline, from the three discovery sources through the mention-count ranking and the filtering criteria to the final set. Table 3.1 lists the

packages alphabetically with their primary roles in the robotics software ecosystem.

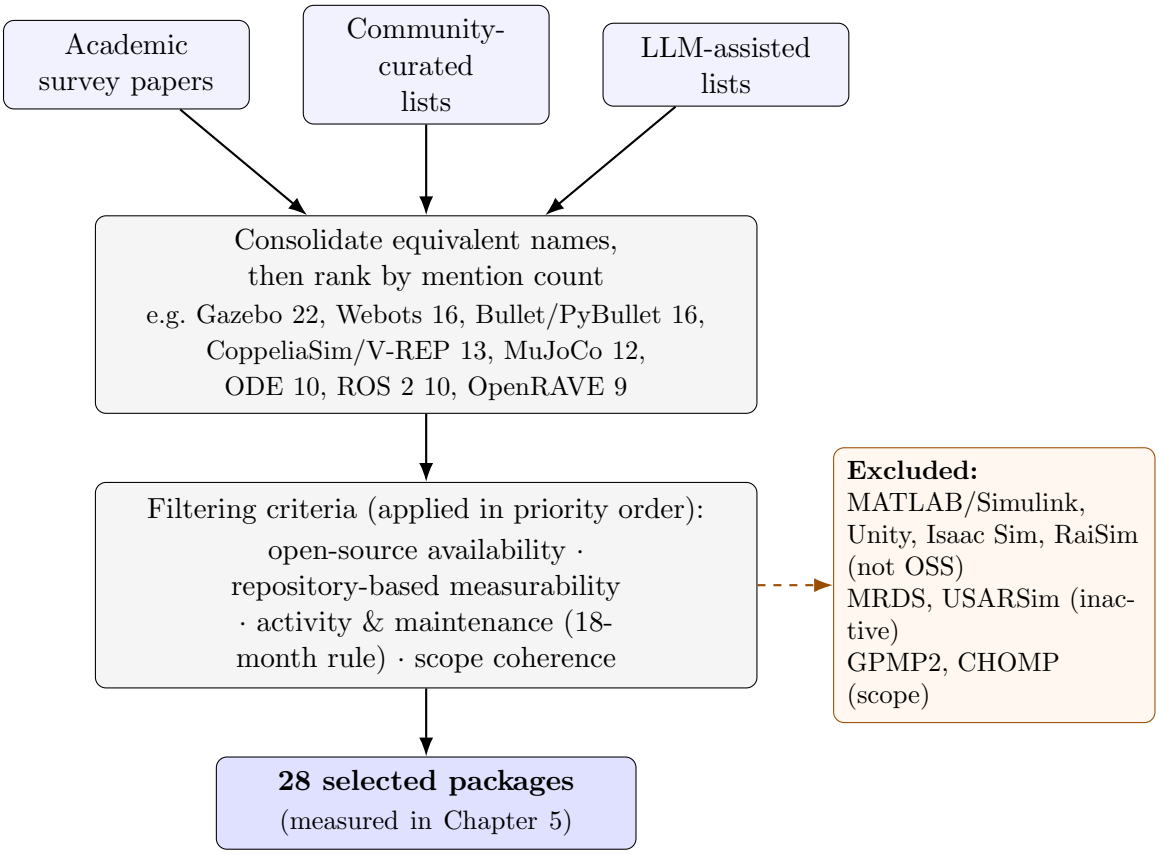

Figure 3.1: Candidate selection pipeline. Packages discovered from three source types are consolidated and ranked by mention count, then reduced by the filtering criteria of Section 3.5 to the 28 measured packages. Mention counts are source-appearance signals only (Section 3.4); representative excluded tools and the reason for each exclusion are shown at right.

The role column is necessary because the packages are not direct substitutes. ROS 2, MoveIt!, and OMPL are boundary cases because they are not full simulators in the same sense as Gazebo or Webots; Chapter 4 provides the detailed category and commonality analysis used to interpret them.

Table 3.1: Final software set with primary roles.

Software	**Primary Role**
AirSim	Simulator (autonomous vehicles)
Bullet/PyBullet	Physics engine
CARLA	Simulator (autonomous driving)
CoppeliaSim (V-REP)	Simulator (general robotics)
DART	Dynamics library
Drake	Toolbox (dynamics, planning, control)
Flightmare	Simulator (quadrotor)
Gazebo	Simulator (general robotics)
GraspIt!	Manipulation (grasping)
Habitat	Simulator (embodied AI)
MORSE	Simulator (academic robotics)
MoveIt!	Manipulation (motion planning)
MRPT	Library (mobile robotics)
MuJoCo	Physics engine
Newton Dynamics	Physics engine
ODE (Open Dynamics Engine)	Physics engine
OMPL	Motion planning library
OpenRAVE	Planning framework
OpenRTM-aist	Middleware (RT components)
OROCOS	Control framework
PhysX (open-source SDK)	Physics engine
Pinocchio	Dynamics library
Project Chrono	Multi-physics simulation
ROS 2	Middleware and SDK
Simbody	Multibody dynamics library
SOFA	Simulation framework
Webots	Simulator (general robotics)
YARP	Middleware (humanoid robotics)

3.7 Measurement Procedure

The measurement procedure follows the Domain X method for collecting structured evidence about software packages in a selected domain (Smith et al., 2021). For each of the 28 selected packages, the study gathered source code, documentation, publications where applicable, repository metadata, issue-tracker evidence, release information, and installation evidence. The measurement record identifies the specific repository or repositories measured, since some robotics projects are distributed across multiple repositories.

3.7.1 Measurement Template

The primary measurement instrument is the measurement template from Appendix B of Smith et al. (2021); the version used in this study is reproduced in Appendix C. The template contains approximately 108 fields when supplementary entries (free-text "additional comments" fields after each section and the GitHub-style and GitStats/SCC repository metric panels) are counted alongside the substantive questions. The substantive questions are grouped into the following sections:

1. **Summary Information** (17 questions): software name, URL, affiliation, purpose, number of developers, funding, initial release date, last commit date, status, license, platforms, software category, development model, publications, source code URL, programming languages, and evidence of performance consideration.

2. **Installability** (14 questions): installation instructions (presence, single-document

organization, linear ordering, completeness, and assumed pre-installed dependencies), compatible operating system versions, installation automation, error handling, installation validation procedure, number of installation steps, operating system used during measurement, dependencies (presence of guidance and required version listing), and uninstall behaviour (problems and residue).

3. **Correctness and Verifiability** (8 questions): requirements or theory documentation, correctness techniques, tutorial availability and quality, unit tests, and continuous integration evidence.

4. **Surface Reliability** (5 questions): installation and tutorial breakage, error messages, and recoverability.

5. **Surface Robustness** (2 questions): handling of unexpected input and edge cases.

6. **Surface Usability** (4 questions): tutorial, user manual, user characteristics, support model.

7. **Maintainability** (7 questions): version number, contribution guidance, available artifacts, issue tracking, issue closure rate, comment percentage, and version control.

8. **Reusability** (2 questions): number of code files and API documentation.

9. **Surface Understandability** (8 questions): code formatting, coding standards, identifier quality, constant usage, comment clarity, algorithm references, parameter ordering, and modularization.

10. **Visibility and Transparency** (4 questions): development process definition, process documentation, development environment documentation, and release notes.

Applying the same template to every package ensures that comparisons are based on a consistent set of observations rather than ad hoc impressions. Each quality section concludes with an overall impression score on a scale of 1 to 10, assigned using the scoring guidelines of the impression calculator in Appendix C of Smith et al. (2021). The impression scores summarize more than the recorded checklist answers alone: they also reflect observations made during measurement, such as the quality and currency of documentation, the clarity of error messages, and the overall installation and tutorial experience, which the mostly binary checklist fields do not capture. Two packages with identical checklist answers can therefore receive different impression scores; Chapter 9 discusses the subjectivity this introduces.

3.7.2 Repository Metrics

Repository metrics are collected separately from the quality impression scores. Three categories of repository data are gathered:

1. **GitHub-style metrics:** stars, forks, watchers, open pull requests, closed pull requests, number of developers, open issues, closed issues, initial release date, and last commit date.

2. **GitStats-style metrics:** number of text-based files, number of binary files, total lines, lines added, lines deleted, total commits, commits by year (last 5 years), and commits by month (last 12 months).

3. **SCC-style metrics:** number of text-based files, total lines, code lines, comment lines, and blank lines.

From these raw data, three processed measures are calculated:

1. **Alive/dead status:** alive is defined as the presence of commits or version releases within the last 18 months from the measurement date.

2. **Percentage of identified issues that are closed:** computed as closed issues divided by total identified issues.

3. **Percentage of code that is comments:** computed as comment lines divided by total lines.

3.7.3 Measurement Execution

The Domain X methodology recommends performing installation-based measurements in a controlled environment, typically a virtual machine, to ensure a clean and reproducible starting state (Smith et al., 2021). For this report, installation measurements were performed on a Linux-based virtual machine. The operating system, installation steps, dependencies, and any issues encountered were recorded for each package.

The methodology also recommends a time budget to keep the overall measurement workload feasible. The full assessment is estimated at roughly 173 person-hours per domain, against a stated goal of about one person-month (160 hours), with 1 to 4 hours allocated per package for template completion and up to 2 hours for installation (Smith et al., 2021). Measurement was conducted during May 2026. All

time-sensitive data (last commit dates, issue counts, stars, forks, etc.) should be understood as snapshots taken during this period.

Several measurement questions require explicit interpretation rules:

- **Last commit date** is recorded at the time of measurement and may change for active repositories.
- **Percentage of identified issues that are closed** is computed consistently as closed issues divided by total identified issues, using GitHub issue data where available.
- **Evidence that performance was considered** is assessed by searching software artifacts for explicit references to speed, storage, throughput, performance optimization, parallelism, multi-core processing, or similar considerations.
- **Percentage of code that is comments** is computed using the SCC tool's line-based method, as comment lines divided by total lines.

3.7.4 Ranking with AHP

After the quality measures are collected, the Domain X methodology uses the Analytic Hierarchy Process (AHP) to support ranking across the nine quality categories (Saaty, 1980). AHP is a decision-making technique that uses pairwise comparisons between alternatives to calculate relative scores. It organizes multiple criteria in a hierarchical structure and enables the combination of individual quality rankings into an overall ranking based on the relative priorities assigned to each quality.

In this report, AHP would be applied to the nine quality impression scores for

the 28 software packages and would produce a ranking that depends on the chosen criterion weights. Sensitivity analysis would then assess whether the ranking is stable under reasonable variations in those weights. An elicited-weight AHP ranking is not computed here, because a defensible criterion-weighting elicitation has not been completed at the time of writing; Chapter 5 therefore reports the per-package quality-mean ranking and a comparison with community indicators, together with an illustrative equal-weight AHP computation that demonstrates the weighting step.

3.8 Commonality Analysis Procedure

The commonality analysis identifies shared and variable capabilities across the selected software packages, treating them as members of a software family as defined by Parnas (1976) and Weiss (1998). The analysis supplements repository-based metrics by describing what the packages do and how their roles differ, providing context for interpreting the measurement results reported in Chapter 5.

Following the Domain X methodology, the analysis proceeds in six steps:

1. **Write an introduction** to the domain and the software family.

2. **Write an overview of domain knowledge,** summarizing the key concepts and terminology relevant to MPS software.

3. **List commonalities:** identify goals, assumptions, functions, artifacts, and domain concepts that recur across multiple family members. For example, many of the selected packages share the goal of simulating robot dynamics, the assumption that robot models are described using standard formats (such as URDF or SDF), and the function of providing an API for programmatic control.

4. **List variabilities:** identify dimensions on which family members differ. These include software role (simulator, physics engine, planning library, middleware), supported platforms, licensing, programming language interfaces, documentation style, and intended use cases.

5. **List parameters of variation:** for each variability, describe the possible values. For example, the software role parameter can take values such as general-purpose simulator, domain-specific simulator, physics engine, dynamics library, motion planning library, manipulation framework, or middleware.

6. **Add terminology, definitions, and acronyms** that are standard in the domain, to ensure that the analysis is accessible to readers outside robotics.

The procedure first groups the selected packages into broad categories based on their primary roles: robotics simulators, physics and dynamics engines, and planning or middleware tools. It then identifies capabilities that appear across multiple categories, such as support for robot model representation, simulated environments, motion planning workflows, dynamics computation, and integration with robotics ecosystems.

Because the selected packages span multiple roles, the commonality analysis does not force all tools into a single comparison template. Its purpose is to describe shared concerns and meaningful differences, not to claim that every package provides identical functionality. The analysis is based on checked evidence from documentation, repositories, publications, and other reliable project materials. Capabilities that could not be verified against these materials are not reported.

3.9 Interview Component

The Domain X methodology includes a developer survey component designed to capture information that is not visible in public repositories: the development process as experienced by practitioners, the pain points they encounter, their perceptions of software quality threats, and the strategies they use to address those threats (Smith et al., 2021). The methodology prescribes semi-structured interviews based on a list of 20 questions covering developer background, project organization, user understanding, development obstacles (past and present), documentation practices, and specific software qualities including maintainability, understandability, usability, and reproducibility.

For this report, the interview component requires research ethics approval from the McMaster Research Ethics Board (MREB), with the application submitted through the MacREM online system, before recruitment or data collection. The planned interview participants are software developers, maintainers, or programmers associated with the studied packages. Approved ethics materials and the interview protocol will be archived in Appendix E.

Chapter 7 records the planned reporting structure for the interview component. Until ethics approval, recruitment, and analysis are complete, that chapter does not report developer findings; it identifies RQ6–RQ9 as questions that require interview evidence.

The methodology recommends sending interview requests to developers from all packages in the study set to avoid selection bias. Prior applications of the methodology report response rates roughly in the 15%–30% range (Smith et al., 2021, 2024a);

the exact rate for this report will depend on outreach timing and the response practices of the contacted developers. Interview contacts are typically identified through project websites, code repositories, publications, or institutional biography pages.

Chapter 4

Selected Software and Commonality Analysis

Chapter 3 explains how the final 28 packages were selected. This chapter characterizes that set for RQ1 and gives the commonality context used later when interpreting the RQ2 measurements. It first summarizes the packages, then groups them by primary role, identifies shared capabilities and major variation points, records parameters of variation and binding times, and closes with boundary cases that do not fit cleanly into one category.

The commonality analysis follows the approach described by Weiss (1998) and is based on evidence from project documentation, repositories, publications, and other reliable sources. Capabilities are reported only where they could be verified against these materials.

4.1 Overview of Selected Software

The 28 software packages selected through the discovery and filtering process described in Chapter 3 represent a cross-section of the MPS software ecosystem. They come from a mix of development settings. Academic research groups account for several entries (for example, Drake, DART, and OMPL). Open-source foundations and companies steward others, notably ROS 2 and Gazebo through Open Robotics, with the ROS-Industrial Consortium coordinating related industrial applications. Corporate research labs contribute MuJoCo (Google DeepMind), AirSim (Microsoft Research), and Habitat (Meta AI Research). Smaller communities or individual contributors maintain projects such as ODE and Newton Dynamics.

The packages span a period of active development from 2001 (ODE's initial release) to the present, with most showing recent repository activity as of the May 2026 measurement date. All 28 packages have public source repositories. Twenty-seven use standard open-source licenses; CoppeliaSim follows a mixed model that combines a GPL-licensed open-source core with commercial and educational distributions. BSD is the most common license (11 packages), followed by Apache 2.0 (5) and MIT (4); the remainder use LGPL, GPL, zlib, or mixed/dual schemes, as detailed in Table 5.1 in Chapter 5. The majority of packages are written primarily in C++ with Python bindings, reflecting the dominant languages in the robotics software community. All 28 packages support Linux. Most packages also support Windows or macOS, with cross-platform support varying by project (see Chapter 5 for details).

This chapter does not report measurement scores or quality rankings; those appear in Chapter 5. Its purpose is to describe what the selected packages do, how they relate to one another, and what dimensions of variation characterize the software family, so

that the measurement results in Chapter 5 can be interpreted in context.

4.2 Software Categories

The selected packages can be grouped into three broad categories based on their primary function within a robotics workflow. These categories are not strict boundaries (some packages span multiple roles) but they provide a useful structure for the commonality analysis.

4.2.1 Robotics Simulators

Robotics simulators provide virtual environments for representing robots, worlds, sensors, and actuators. They enable researchers and engineers to test algorithms, validate designs, and collect synthetic data before deploying to physical hardware. Packages in this category typically offer graphical rendering, physics simulation, sensor models, and programmatic interfaces for controlling simulated robots.

The following packages from the selected set are primarily classified as robotics simulators:

- **Gazebo** (including the Ignition/Gazebo Sim lineage) is a long-established open-source 3D robotics simulator developed by Open Robotics. It provides physics simulation through multiple backends, rendering, sensor models, and ROS integration.
- **Webots** is a multi-platform robot simulator maintained by Cyberbotics. It supports modelling, programming, and simulating robots, vehicles, and mechanical systems, with a large library of pre-built robot models.

- **CoppeliaSim** (formerly V-REP) is a versatile robot simulation framework developed by Coppelia Robotics. It supports rapid algorithm development, factory automation simulation, prototyping, verification, and digital twin applications.

- **AirSim** is an open-source simulator for autonomous vehicles (drones and ground vehicles) built on Unreal Engine and developed by Microsoft Research. The measured repository is `microsoft/AirSim`; under the May 2026 18-month activity rule it is classified as alive, with a last recorded commit on 2026-03-15. This label should be read narrowly: the repository README has announced since 2022 that development has ended ("no further updates will be made, effective immediately") and points users toward Project AirSim, and the few commits recorded after 2022 are README-level changes. The alive label is therefore a repository snapshot under the activity rule, not a claim about continuing stewardship of the original AirSim line.

- **CARLA** is an open-source autonomous driving simulator built on Unreal Engine and developed by the Computer Vision Center in Barcelona and Intel. It provides photorealistic urban environments and a broad sensor suite for autonomous driving research.

- **MORSE** (Modular OpenRobots Simulation Engine) is a generic academic robotic simulator based on Blender and the Bullet physics engine, designed to remain middleware-agnostic across ROS, YARP, and other frameworks.

- **Habitat** is a 3D simulator for embodied AI research developed by Meta AI Research. It supports training and evaluation of embodied AI agents in photorealistic indoor environments.

- **Flightmare** is a flexible modular quadrotor simulator developed by the Robotics and Perception Group at the University of Zurich and ETH Zurich. It separates rendering from the RL environment for flexible configuration between fast and photorealistic modes.

Several additional packages provide simulation capabilities as part of broader functionality. Drake includes simulation features within its model-based design toolbox. MRPT includes a 3D simulation environment alongside its mobile robotics libraries. SOFA provides interactive simulation for medical and soft robotics applications. These packages may be considered simulators in some contexts but serve broader roles.

4.2.2 Physics Engines and Dynamics Libraries

Physics engines and dynamics libraries provide the computational foundation for simulating rigid-body and multibody dynamics, collision detection, and contact resolution. They are often used as backends by higher-level simulators but can also be used directly by researchers and developers.

The following packages are primarily classified as physics engines or dynamics libraries:

- **Bullet/PyBullet** is a widely used open-source physics engine for simulation, robotics, and deep reinforcement learning. PyBullet provides Python bindings that have made it popular in the machine learning community.

- **MuJoCo** (Multi-Joint dynamics with Contact) is a general-purpose physics simulator maintained by Google DeepMind. It is designed for fast and accurate

simulation of articulated structures and is widely used in robotics and reinforcement learning research.

- **ODE** (Open Dynamics Engine) is one of the earliest open-source libraries for simulating rigid body dynamics. It has been used extensively in robotics and served as a physics backend for Gazebo Classic.

- **PhysX** is NVIDIA's real-time physics engine SDK. The open-source SDK provides capabilities for simulating rigid bodies, particles, vehicles, and deformable bodies, primarily targeted at real-time simulation applications.

- **DART** (Dynamic Animation and Robotics Toolkit) is a C++ physics engine and kinematics/dynamics library developed at Georgia Tech and other institutions. It is designed for stable simulation in manipulation and locomotion research.

- **Simbody** is a multibody dynamics library developed at Stanford University for simulating articulated biomechanical and mechanical systems. It is the physics engine underlying OpenSim (biomedical simulation software).

- **Pinocchio** is a fast and flexible C++ library for rigid-body dynamics computations and their analytical derivatives, developed primarily at Inria and LAAS-CNRS. It implements state-of-the-art algorithms for kinematics, dynamics, and Jacobians.

- **Project Chrono** is an open-source multi-physics simulation library developed

at the University of Wisconsin-Madison and the University of Parma. It supports multibody dynamics, finite element analysis, vehicle dynamics, and GPU-accelerated simulation.

- **Newton Dynamics** is a cross-platform physics simulation engine optimized for real-time deterministic rigid-body simulation. It is used in game development and robotics simulation.

- **SOFA** (Simulation Open Framework Architecture) is an open-source real-time simulation framework developed primarily at Inria. It focuses on deformable object simulation, finite element methods, and haptic feedback, with applications in surgical simulation and soft robotics.

4.2.3 Motion Planning, Middleware, and Manipulation Tools

Motion planning libraries, robotics middleware, and manipulation tools cover parts of the workflow outside full-world simulation: planning algorithms, communication layers, grasp planning, and system integration. They are included because MPS workflows typically combine planning, simulation, and execution rather than using simulators alone.

The following packages are primarily classified as planning or middleware tools:

- **ROS 2** (Robot Operating System 2) is an open-source software development kit and middleware framework for building robot applications. It provides communication infrastructure (based on DDS), hardware abstraction, drivers, libraries, visualization tools, and a large ecosystem of community packages. ROS 2 is the successor to ROS 1 and represents a major redesign with emphasis on real-time

control, multi-robot deployment, and production use.

- **MoveIt!** refers to the active MoveIt 2 generation for ROS 2 in this report. It is an open-source robotics manipulation framework that provides motion planning, kinematics, collision checking, trajectory execution, and perception integration for robot arms and manipulators, integrating OMPL and other planners as plugins.

- **OMPL** (Open Motion Planning Library) is an open-source C++ library for sampling-based motion planning algorithms developed at Rice University. It provides implementations of widely used planners such as PRM, RRT, RRT*, and EST, and is the default planning backend for MoveIt!.

- **OpenRAVE** (Open Robotics Automation Virtual Environment) is an open-source robotics planning framework focused on simulation and analysis of kinematic and geometric information for motion planning, particularly for manipulation tasks.

- **Drake** is an open-source C++ and Python toolbox for model-based design and verification of robotics systems, developed by MIT and the Toyota Research Institute (TRI). It covers multibody dynamics, trajectory optimization, model predictive control, perception, and manipulation planning.

- **OROCOS** (Open RObot COntrol Software) is an open-source C++ framework for real-time robot control developed at KU Leuven. Its major components include the Real-Time Toolkit for component-based hard real-time software and the Kinematics and Dynamics Library.

- **MRPT** (Mobile Robot Programming Toolkit) is an open-source cross-platform C++ library for mobile robotics research, providing algorithms for SLAM, computer vision, motion planning, and sensor data processing.
- **YARP** (Yet Another Robot Platform) is open-source middleware for humanoid and general robotics, developed at the Istituto Italiano di Tecnologia. It provides communication infrastructure for distributed robot systems and is the primary middleware for the iCub humanoid robot platform.
- **OpenRTM-aist** is an implementation of the OMG-standardized Robot Technology Component standard, developed at AIST in Japan. It provides a component-based framework for creating and connecting robot software modules.
- **GraspIt!** is a simulation environment for robotic grasping research developed at Columbia University. It supports loading robot hand models and 3D object models to plan, evaluate, and visualize grasps.

4.3 Commonalities

Despite the diversity of the selected packages, several capabilities and concerns recur across the software family. These commonalities are organized around the abstraction of a typical robotics software workflow: model representation, computation, interaction, and documentation. Figure 4.1 presents the family as a commonality/variability feature model: the recurring common concerns (C1–C7) are detailed in this section, and the variable features (V1–V9) in Section 4.4.

C1: Robot Model Representation. All selected packages provide or consume some form of robot model representation. Simulators and physics engines represent

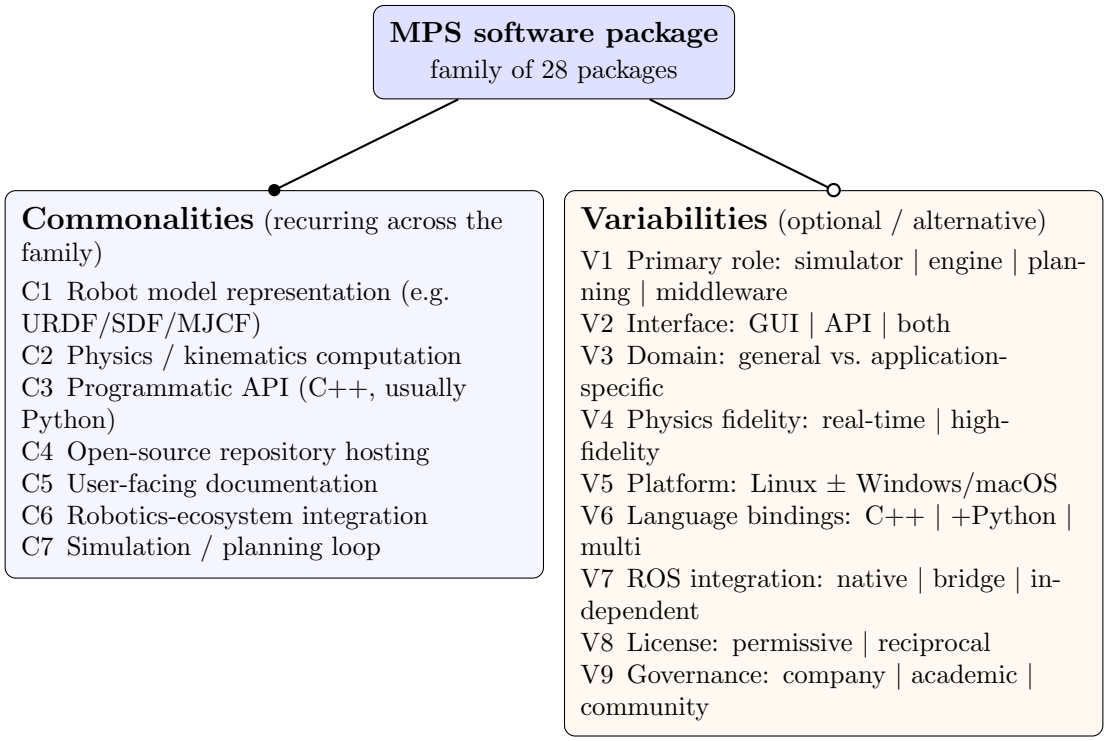

Figure 4.1: Commonality/variability feature model for the 28-package MPS software family. Filled circle (•) marks commonalities, the concerns that recur across the family; open circle (∘) marks variabilities, the points on which packages differ. Some commonalities are near-universal (every package exposes a programmatic API, hosts an open-source repository, and ships documentation), while others (physics/kinematics computation, ecosystem integration, a simulation or planning loop) are shared by most but not all packages, as detailed in Section 4.3. Variabilities V1–V9 and their binding times appear in Section 4.4 and Table 4.1.

robot kinematics and dynamics through internal model structures (often supporting standard formats such as URDF, SDF, or MJCF). Motion planning libraries operate on kinematic and geometric descriptions of robots and environments. Middleware packages define standardized message formats for communicating robot state.

C2: Physics or Kinematics Computation. The packages in the simulator and physics engine categories share the capability of computing robot motion under physical constraints. This includes forward and inverse kinematics, rigid-body dynamics, collision detection, and contact resolution. Even middleware and planning tools typically include kinematic computation capabilities or depend on libraries that provide them.

C3: Programmatic Interface. All 28 packages provide a programmatic interface (typically a C++ API, often with Python bindings) that allows developers to control the software, configure simulations or planners, and integrate the package into larger robotics workflows.

C4: Open-Source Repository Hosting. All 28 packages use public version control hosting (GitHub for 27 of 28 packages) with visible commit history, issue tracking, and pull request management. This shared infrastructure is both a selection criterion and a commonality of the selected set.

C5: Documentation. All packages provide some form of user-facing documentation, typically including installation instructions, a getting-started tutorial, and API reference material. The depth, currency, and completeness of this documentation vary considerably, as discussed in Chapter 5.

C6: Integration with Robotics Ecosystems. Most packages provide or support integration with broader robotics ecosystems. ROS 2 integration is the most

common pattern, with many simulators offering ROS 2 interfaces for publishing sensor data and accepting control commands. Some packages (such as Gazebo and MoveIt!) are deeply embedded in the ROS ecosystem, while others maintain their own integration frameworks.

C7: Simulation or Planning Loop. Simulators and planners share a common architectural pattern: a loop that iteratively updates the system state. In simulators, this is the physics stepping loop that advances the simulation in time. In planners, it is the search or optimization loop that explores the configuration space to find a valid path.

4.4 Variabilities

The selected packages differ along several important dimensions. These variabilities are not deficiencies; they reflect the different roles that packages play within the robotics software ecosystem.

V1: Primary Software Role. The most fundamental variability is the role a package plays in a robotics workflow. As described in Section 4.2, packages range from full simulation environments (Gazebo, Webots, CoppeliaSim) to physics engines (Bullet, MuJoCo, ODE) to planning libraries (OMPL) to middleware frameworks (ROS 2, YARP).

V2: User Interface Model. Packages differ in how users interact with them. Some provide graphical user interfaces with 3D rendering, scene editors, and interactive controls (Gazebo, Webots, CoppeliaSim). Others are primarily libraries or APIs intended for programmatic use (OMPL, Pinocchio, ODE). Some provide both (Drake, MRPT).

V3: Domain Specificity. Some packages are general-purpose robotics simulators or engines (Gazebo, Bullet, MuJoCo), while others target specific application domains: CARLA and AirSim for autonomous vehicles, Flightmare for quadrotor flight, Habitat for embodied AI in indoor environments, SOFA for medical simulation, and GraspIt! for robotic grasping.

V4: Physics Fidelity and Approach. Physics engines differ in their simulation approach and fidelity. Some prioritize speed for real-time or reinforcement learning applications (MuJoCo, Bullet), while others emphasize physical accuracy for engineering analysis (Project Chrono, Simbody). Differences include the choice of constraint solver, collision detection algorithm, integration method, and support for deformable bodies or fluid dynamics.

V5: Platform Support. All packages support Linux, and support for other platforms varies. Some packages provide first-class support for Windows and macOS (Webots, CoppeliaSim, Bullet), while others are primarily developed for Linux with limited cross-platform support.

V6: Programming Language Interfaces. C++ is the dominant implementation language across the selected packages, but packages differ in the range and quality of language bindings they provide. Python bindings are increasingly common because Python has become central to robotics research and machine learning. Fewer packages provide interfaces for languages such as MATLAB, Java, or Rust.

V7: ROS Integration Depth. Packages vary in their relationship to the ROS ecosystem. Some (Gazebo, MoveIt!) are tightly integrated and depend on ROS for core functionality. Others (Webots, CARLA) provide ROS bridges as optional components. Some (YARP, OROCOS) are alternatives to ROS that provide their

own middleware infrastructure.

V8: License Model. Although all packages are open source, they use different licenses. The most common are BSD (2-clause and 3-clause), Apache 2.0, MIT, and LGPL. License choice affects how the software can be reused and integrated into other projects.

V9: Community and Governance. Packages differ in their development governance. Some are stewarded by established organizations (Open Robotics for Gazebo and ROS 2, Google DeepMind for MuJoCo, NVIDIA for PhysX), some by academic research groups (Drake at MIT/TRI, OMPL at Rice, DART at Georgia Tech), and some by individual maintainers or small communities (ODE, Newton Dynamics, GraspIt!).

4.5 Parameters of Variation

For each variability identified in Section 4.4, Table 4.1 summarizes the parameters of variation and their typical binding times. Binding time indicates when a particular choice is fixed: at specification time (when the software is designed), build time (when it is compiled), configuration time (when it is set up), or run time (during execution).

The binding times reflect when a user or integrator can make choices. A package's primary role and governance model are fixed at specification time—they are inherent to the software's design. Platform support is largely determined at build time, though some packages allow configuration-time choices. ROS integration depth can often be chosen at configuration time, as many packages offer optional ROS bridges. Physics fidelity involves trade-offs that may be configured at both specification time (choice of engine) and run time (choice of solver parameters or time step).

Table 4.1: Parameters of variation and binding times.

Variability	**Parameters of Variation**	**Binding Time**
V1: Primary role	Simulator, physics engine, dynamics library, motion planning library, manipulation framework, middleware, multi-purpose toolbox	Specification time
V2: User interface	Graphical (3D scene, editor), programmatic (API only), both	Specification time
V3: Domain specificity	General-purpose robotics, autonomous driving, aerial vehicles, embodied AI, medical/soft robotics, grasping, mobile robotics	Specification time
V4: Physics fidelity	Real-time (game-quality), high-fidelity (engineering-quality), configurable trade-off	Specification / configuration time
V5: Platform support	Linux-only, Linux + Windows, Linux + Windows + macOS	Specification / build time
V6: Language bindings	C++ only, C++ with Python, multi-language (Python, MATLAB, Java, etc.)	Build / configuration time
V7: ROS integration	Native (ROS-dependent), bridge (optional ROS interface), independent (separate middleware), alternative (non-ROS middleware)	Configuration time
V8: License	Permissive (BSD, MIT, Apache 2.0), reciprocal (LGPL, GPL)	Specification time
V9: Governance	Single institution or company, academic consortium, community-maintained, individual developer	Specification time

4.6 Boundary Cases

Several of the selected packages occupy intermediate positions in the classification presented above. These boundary cases are not classification errors; they illustrate the interconnected nature of the robotics software ecosystem and the difficulty of drawing sharp boundaries between categories.

ROS 2 is classified as middleware in this analysis, but its scope extends well beyond communication infrastructure. ROS 2 provides an SDK with libraries, drivers, visualization tools (RViz), simulation integration (through Gazebo), and a package management system. It is the de facto integration layer for robotics software components, and many other packages in the selected set assume it as their runtime environment. Its inclusion in a study of MPS software is justified by its central role in the workflows that combine these capabilities.

Drake spans multiple categories. It provides simulation capabilities without being a full simulator in the sense of Gazebo or Webots. It includes sophisticated multibody dynamics without being solely a physics engine. It offers trajectory optimization and model predictive control without being only a planning library. Drake is best understood as a model-based design toolbox that integrates dynamics, planning, and control into a unified framework.

MoveIt! and **OMPL** are closely related but serve different roles. OMPL is a general-purpose motion planning library that can be used independently. MoveIt! is a manipulation framework that integrates OMPL (among other planners) with kinematics, collision checking, and trajectory execution for robot arms. OMPL is classified as a motion planning library and MoveIt! as a manipulation framework, but the boundary between them reflects architectural layering rather than functional

separation.

SOFA is grouped with physics engines and dynamics libraries here because the measured materials emphasize deformable-body simulation, finite element methods, and haptic physical interaction. The package also presents itself as a simulation framework, especially for medical and soft-robotics applications. For that reason, it is treated as a boundary case rather than as a direct peer of rigid-body engines such as ODE or Bullet.

These boundary cases are not treated as measurement anomalies. In the chapters that follow, measurements are reported for each package individually, and comparisons are interpreted in the context of each package's role within the robotics software ecosystem.

Chapter 5

Ranking Projects by Quality Measure

This chapter answers RQ2. It ranks the 28 selected software packages across the nine Domain X quality categories and then compares that ranking with community-facing indicators of visibility and adoption. The data were collected in May 2026 and capture the repository state, documentation, and project activity visible at that time. These are public-artifact measurements rather than full product evaluations; Chapter 3 states the rules under which the repository metrics, quality scores, and community indicators were collected and are to be interpreted.

The quality measurements follow the measurement template from Appendix B of Smith et al. (2021) and cover the nine quality categories described in Section 2.4. The chapter first presents summary metadata, then reports the per-category results, repository metrics, the quality-mean ranking, and the comparison with community indicators.

5.1 Repository and Project Metadata

Table 5.1 presents summary metadata for all 28 packages, including the primary development organization, license, initial release date, and primary programming languages. Platform support is summarized in the narrative following the table. The packages are listed alphabetically.

All 28 packages have publicly accessible source repositories. Twenty-seven follow an open-source development model under standard permissive or copyleft licenses; CoppeliaSim is the exception, with a mixed model that combines a GPL-licensed open-source codebase with commercial and educational licenses for its packaged distribution. The most common licenses are BSD (11 packages), Apache 2.0 (5), and MIT (4). The packages span more than two decades of development history: ODE is the oldest, with its initial release in 2001, while PhysX's open-source SDK was released in 2022.

C++ is the dominant implementation language, listed as a primary or major language by 25 of 28 packages in the measurement records. Three packages fall outside that count: MORSE (primary languages: C and Python), ODE (primary language listed as "unclear" because the public source tree mixes C and C++ files without a single declared primary), and ROS 2 (also "unclear" because the ROS 2 organization spans hundreds of repositories with different primary languages, although the core communication libraries themselves are in C++). Python is a primary or secondary language in most packages, reflecting its growing role in robotics research and its use for scripting, bindings, and machine learning integration. All 28 packages support Linux. Twenty-four packages support Windows and 23 support macOS.

Table 5.1: Summary metadata for the 28 selected software packages (May 2026). The Initial Release column records the date associated with the measured public repository: for packages with a single continuous open-source history this is the first public release, while for packages that were renamed, re-licensed, or migrated to open source after an earlier proprietary or pre-release phase (for example CoppeliaSim, MuJoCo, Webots, Gazebo, PhysX), the date reflects that later event rather than the project's original creation.

Software	**Primary Affiliation**	**License**	**Initial Release**	**Primary Languages**
AirSim	Microsoft Research	MIT	2017	C++, Python
Bullet/ PyBullet	Community	zlib	2006	C++, Python
CARLA	CVC Barcelona, Intel	MIT	2017	C++, Python
CoppeliaSim	Coppelia Robotics	GPL + comm.	2019	C++, Python
DART	Georgia Tech	BSD	2011	C++, Python
Drake	TRI, MIT	BSD	2010	C++, Python
Flightmare	U. Zurich, ETH	MIT	2020	C++, Python
Gazebo	Open Robotics	Apache-2.0	2014	C++, Python
GraspIt!	Columbia U.	GPL v3+	2010	C++, Python
Habitat	Meta AI Research	MIT	2019	C++, Python
MORSE	LAAS-CNRS	BSD	2009	C, Python
MoveIt!	PickNik Robotics	BSD	2011	C++, Python
MRPT	U. Almería	BSD	2010	C++, Python
MuJoCo	Google DeepMind	Apache-2.0	2021	C++, Python
Newton Dynamics	Independent	zlib	2013	C++, C
ODE	Independent	BSD/ LGPL	2001	unclear
OMPL	Rice University	BSD	2010	C++, Python
OpenRAVE	CMU	LGPL + A2.0	2009	C++, Python
OpenRTM-aist	AIST (Japan)	LGPL-2.1	2005	C++, Python

As of the May 2026 measurement date, 24 of the 28 packages (86%) were classified as alive under the Domain X 18-month activity rule, which defines alive as the presence of any commit or release in the preceding 18 months. Four packages were classified as dead: OpenRAVE (last commit August 2024), MORSE (last commit March 2022), GraspIt! (last commit July 2020), and Flightmare (last commit May 2023). ODE is technically alive under the 18-month rule (last commit 2026-04-14) but its commit activity in the trailing twelve months is sparse (five commits across the year); the analysis that follows treats ODE as a low-activity alive package and discusses its limitations alongside the dead packages where appropriate.

The number of developers (anyone who has made at least one accepted commit) ranges from 6 (CoppeliaSim, Flightmare) to 598 (MoveIt!). The median is approximately 128.5 (the mean of the 14th and 15th values, YARP at 125 and MRPT at 132). The total number of commits across all measured repositories exceeds 240,000. Drake leads with 34,786 commits, followed by SOFA (27,404) and Project Chrono (19,510). These repository-level metrics are snapshots and should not be interpreted as direct measures of project scale or quality without considering factors such as repository age, commit granularity, and whether development activity is concentrated in a single repository.

PhysX requires a particular caveat. The measured repository (`NVIDIA-Omniverse/PhysX`) is the public open-source SDK release of NVIDIA's PhysX engine; the broader production engine and its associated engineering process are developed in an internal NVIDIA codebase that is not publicly accessible. All PhysX scores reported in this chapter, including the small public commit count and the per-quality impression scores, reflect only the public SDK surface as it appeared at the measurement date.

They should not be read as an assessment of the PhysX engine as a product, or of NVIDIA's internal development practices. Where PhysX scores appear lower than peers in subsequent sections, the gap reflects what is publicly observable rather than a verdict on the engine itself.

Two further repository-identity caveats apply. The OROCOS measurements are based on the `orocos_kinematics_dynamics` repository, which hosts the Kinematics and Dynamics Library (KDL); the project's other components, such as the Real-Time Toolkit, live in separate repositories and are not represented in the OROCOS code-size, commit, or activity figures. For Newton Dynamics, the measured repository is the maintainer's current development home, created in January 2026 when development moved away from the project's long-standing original repository. Its star and fork counts therefore describe a repository that was only a few months old at measurement time, not the community attention the project accumulated over two decades; Section 5.13 repeats this caveat where it affects interpretation.

5.2 Installability

Installability measures the effort required to install and uninstall software in a specified environment. All installation measurements were performed on Linux-based virtual machines. Table 5.2 summarizes key installability indicators across the selected packages.

Table 5.2: Key installability indicators across 28 packages.

Indicator	**Packages**	**%**
Installation instructions present	28	100%
Instructions in a single document or page	26	93%
Instructions are linear (single series of steps)	26	93%
Instructions assume no pre-installed dependencies	22	79%
Compatible OS versions listed	17	61%
Installation automation provided (script, makefile, etc.)	28	100%
Installation validation procedure available	27	96%
Dependency installation instructions provided	21	75%
Required package versions listed	19	68%
No problems caused by uninstall (where available)	27	96%

All 28 packages provide installation instructions and some form of installation automation (makefiles, scripts, package managers, or container definitions).

The number of installation steps ranged from 1 to 11, with a median of 2. The number of extra software packages required before or during installation ranged from 0 to 10, with a median of 1. GraspIt! required the most manual effort (11 steps, 10 extra packages), consistent with its dead status and lack of recent maintenance. At the other extreme, ten packages (including Webots, MuJoCo, DART, and Simbody) needed no extra packages at all, and nine could be installed in a single step using package managers or pre-built binaries (MuJoCo, Simbody, and MORSE achieved both).

Installation broke for only 2 of the 28 packages during measurement (OpenRAVE

and YARP; OpenRAVE is also classified as inactive in Section 5.1). In both cases the installation instance was recoverable. No package broke during initial tutorial testing. This 2-of-28 breakage rate compares favourably with the Medical Imaging study, where installation instructions were found for only 16 of 29 packages and three packages could not be installed at all (Smith et al., 2024a).

The overall installability impression scores (on a 1–10 scale) range from 4 to 10, with a mean of 7.4. The top-scoring packages are Gazebo (10), Pinocchio (10), Bullet/PyBullet (9), MORSE (9), and OMPL (9). The lowest-scoring packages are ODE (4), PhysX (4), and GraspIt! (5). For ODE, the low score reflects outdated installation documentation (the official guide dates from 2006 and refers to GNU make, VC6 project files, and manual copying of library files). For PhysX, the low score reflects difficulty locating the correct build documentation. MuJoCo's score of 7, despite a one-step pip installation with no extra packages, reflects a recorded mismatch between installation and verification: running the documented example script after the standard installation failed with a file-not-found error.

5.3 Correctness and Verifiability

Correctness and verifiability assess whether a program matches its specification and the ease with which correctness can be checked. Table 5.3 summarizes key indicators.

Table 5.3: Correctness and verifiability indicators.

Indicator	**Packages**	**%**
Reference to requirements specifications or theory manuals	26	93%
Getting-started tutorial present	28	100%
Tutorial instructions are linear	27	96%
Tutorial provides expected output	25	89%
Tutorial output matches expected output	23	82%
Unit tests available	27	96%
Evidence that performance was considered	28	100%

All 28 packages use at least one technique intended to support correctness. Table 5.3 records unit tests for 27 of 28 packages (96%). Automated testing is recorded separately: 26 of 28 packages have CI or an equivalent setup that exercises tests or related checks on each push or pull request, giving an automated-testing rate of 93%. Assertions in code are used by 23 of 28 packages (82%). Documentation generation tools such as Doxygen or Sphinx are used by 12 packages (43%). Ten packages combine all three techniques: DART, Drake, Habitat, Newton Dynamics, OMPL, OROCOS, PhysX, Pinocchio, Project Chrono, and SOFA all use automated testing, assertions, and a documentation generator together.

All 28 packages provide a getting-started tutorial, and 27 of 28 provide linear tutorial instructions. The tutorial-output check succeeded for 23 of the full 28-package set (82%). Five packages fall outside that positive count: SOFA, Flightmare, and Newton Dynamics do not publish an expected tutorial output, while MuJoCo and OpenRTM-aist had environment-dependent outputs that the standard measurement

procedure could not reproduce.

Unit tests are available for 27 of 28 packages. CoppeliaSim's unit test status is unclear because its test infrastructure is not publicly visible in the same way as the other packages, a consequence of its mixed commercial/open-source licensing model.

The overall correctness and verifiability impression scores range from 5 to 10, with a mean of 7.7. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). The lowest-scoring are GraspIt! (5) and Newton Dynamics (5).

5.4 Surface Reliability

Surface reliability assesses failure-free operation during the limited interactions performed during measurement: installation and initial tutorial use. These are surface checks, not comprehensive reliability experiments.

As noted in Section 5.2, installation broke for 2 of 28 packages (OpenRAVE and YARP). In both cases the installation instance was recoverable; for OpenRAVE a descriptive error message was displayed, while the measurement record does not capture the error output for YARP.

The overall surface reliability impression scores range from 6 to 10, with a mean of 7.8. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). Three packages scored below 7: PhysX (6), GraspIt! (6), and Newton Dynamics (6).

Most packages installed cleanly and their tutorials ran without breakage, which explains the high surface reliability scores. This part of the measurement covers only a shallow interaction (installation plus one tutorial) on a single operating system, and

does not constitute a systematic reliability evaluation.

5.5 Surface Robustness

Surface robustness assesses whether software handles unexpected or unanticipated input reasonably, including edge cases such as malformed input files. All 28 packages were assessed as handling unexpected input reasonably (producing appropriate error messages or graceful degradation rather than crashes or undefined behaviour). For the newline-conversion check (replacing newlines with carriage returns and newlines in plain-text input files), 25 of 28 packages handled the conversion gracefully; the remaining three (CoppeliaSim, ODE, PhysX) were recorded as not applicable because their workflows do not take plain-text input files in this form.

The overall surface robustness impression scores range from 5 to 10, with a mean of 7.4. Gazebo scores highest (10), followed by Drake (9). The lowest-scoring package is GraspIt! (5). The recorded robustness checks themselves separate the packages only weakly: every package was assessed as handling unexpected input reasonably, and the newline-conversion check produced no failures (three packages were recorded as not applicable). The spread in the impression scores therefore comes from the broader observations that feed the impression calculator, such as the quality of error messages and input validation encountered during installation and tutorial use (Section 3.7.1), rather than from the two binary checks alone.

5.6 Surface Usability

Surface usability assesses the presence and quality of user-facing documentation, tutorials, and support infrastructure. It is not a full usability evaluation; no user studies or task-completion experiments were conducted.

All 28 packages provide a getting-started tutorial and a user manual. The Medical Imaging study (Smith et al., 2024a), by comparison, found that only 22 of 29 MI packages (76%) provided a user manual. Only one package in the current measurement (MuJoCo) documents expected user characteristics, which is consistent with findings from other domains where this practice is rare (Smith et al., 2024a,b).

Table 5.4 summarizes the user support models adopted by the study packages. Most packages use multiple support channels. For this table, a support route is counted only when the measurement record treats it as current user or developer support. Historical channels, commercial-inquiry contacts, and indirectly affiliated community spaces remain in the raw records, but they are not included in the totals unless the measured project clearly uses them for support.

Table 5.4: User support models across 28 packages.

Support Channel	**Packages Using**
GitHub Issues / equivalent issue tracker	28
GitHub Discussions	10
Official forum or discussion board	9
Mailing list / Google Group	8
Discord server	6
Documentation website or wiki	28
Stack Exchange / Stack Overflow tag	5
ROS Answers integration	4

GitHub Issues or an equivalent issue tracker appears for all 28 packages; every package also has a dedicated documentation website or wiki. The remaining channels vary: 10 packages use GitHub Discussions, 9 maintain an official forum, 8 use a mailing list or Google Group, 6 maintain a Discord server, 5 use a Stack Exchange or Stack Overflow tag, and 4 integrate with ROS Answers. The pattern partly tracks project age and governance model. Newer packages and those stewarded by companies or large labs tend to favour Discord and GitHub Discussions, while older academic packages more commonly use mailing lists and dedicated forums.

The overall surface usability impression scores range from 5 to 10, with a mean of 7.3. Gazebo (10), Drake (9), MoveIt! (9), and ROS 2 (9) score highest; they pair multiple support channels with thorough user, developer, and API documentation. Newton Dynamics (5) and GraspIt! (5) score lowest.

5.7 Maintainability

Maintainability assesses the effort required to modify a software system or component. Table 5.5 summarizes key maintainability indicators.

Table 5.5: Maintainability indicators across 28 packages.

Indicator	**Count**	**%**
Version number documented	28	100%
Code review / contribution guide	26	93%
Non-code artifacts present	28	100%
Issue tracking via GitHub or equivalent	28	100%
Version control via git (GitHub or BitBucket)	28	100%

All 28 packages use git for version control. Twenty-seven of the 28 host their primary repository on GitHub; ODE is the exception, hosting on BitBucket alongside a GitHub mirror. All 28 use GitHub Issues or the equivalent (BitBucket Issues for ODE) for issue tracking. This uniformity is partly a selection effect, since the Domain X methodology expresses a preference for GitHub-style repositories because they facilitate consistent measurement; it also reflects the near-universal adoption of git and GitHub in the contemporary robotics software community.

The percentage of identified issues that are closed ranges from 34.6% (Flightmare) to 100% (DART, Newton Dynamics), with a mean of 79.1% and a median of 82.0% across the 27 packages that yielded a measurable closure rate (ODE returns no value because the BitBucket API call did not populate this field, as noted under Table 5.8). A high closure rate can indicate active issue management, but it can also reflect a

low rate of new issue reporting or a practice of closing stale issues aggressively.

The percentage of code that consists of comments ranges from 1.1% (Project Chrono) to 28.0% (OROCOS), with a mean of 13.2% across the 27 packages with comparable single-repository measurements. The ROS 2 entry is excluded from this range and mean because its multi-repository structure does not yield a comparable single-repository code-line count (Table 5.8 marks it as not measured rather than as zero). OROCOS (28.0%), OMPL (27.4%), OpenRTM-aist (26.3%), MoveIt! (26.1%), and Simbody (25.3%) lead in comment density. Project Chrono (1.1%), Webots (1.9%), GraspIt! (2.1%), MORSE (2.7%), and Bullet/PyBullet (2.8%) have the lowest comment percentages. The Domain X methodology treats 10% as a threshold for the maintainability impression calculator.

The overall maintainability impression scores range from 4 to 10, with a mean of 7.5. Gazebo (10), Drake (9), MoveIt! (9), Pinocchio (9), and ROS 2 (9) score highest. GraspIt! (4) and Flightmare (4) score lowest, weighed down by their dead or very low activity status and limited contribution infrastructure.

5.8 Reusability

Reusability assesses the extent to which software components can be used in problem solutions other than the one for which they were originally developed. The Domain X template measures reusability through two indicators: the number of code files (as a rough proxy for component granularity) and the presence of API documentation.

The number of code files varies widely across the 27 packages with a comparable single-repository count, from 118 (Flightmare) to 7,823 (Newton Dynamics), with a median of 1,266 (Pinocchio at the midpoint of the sorted list). ROS 2 is again

excluded from this distribution because its multi-repository structure does not yield a single-repository file count comparable to the rest. All 28 packages provide API documentation; where the measurement records identify documentation-generation tools, the most common are Doxygen and Sphinx.

The overall reusability impression scores range from 5 to 10, with a mean of 7.6. Gazebo (10), MuJoCo (9), OMPL (9), Pinocchio (9), ROS 2 (9), and MoveIt! (9) score highest. These packages are designed for integration into larger robotics systems or serve as the foundation that other tools build upon. MORSE (5) and GraspIt! (5) score lowest.

5.9 Surface Understandability

Surface understandability is assessed through a review of 10 randomly selected source files per package, examining code formatting, identifier quality, comment practices, and modularization. This is a shallow check, not a comprehensive code review. One template question, whether parameters appear in the same order across functions, was not recorded during measurement; the discussion and scores below rest on the remaining checks.

Table 5.6 summarizes the surface understandability indicators.

Table 5.6: Surface understandability indicators across 28 packages.

Indicator	**Packages**	**%**
Consistent indentation and formatting style	28	100%
Explicit identification of a coding standard	14	50%
Consistent, distinctive, and meaningful identifiers	28	100%
Hard-coded constants (other than 0 and 1) present	28	100%
Comments are clear and describe what is being done	28	100%
Name/URL of algorithms used is mentioned	28	100%
Code is modularized	28	100%

All 28 packages exhibit consistent indentation and formatting, meaningful identifiers, clear comments, and modularized code organization. These are baseline indicators of professional software development practice, and their universal presence is a positive signal for the MPS domain.

Fourteen of 28 packages (50%) explicitly identify a coding standard. This is stronger than the MI comparison point, where code style guides were rare (Smith et al., 2024a), but it still leaves half of the measured MPS packages without an explicit coding-standard artifact. Packages that do identify a coding standard include Gazebo, Webots, Bullet/PyBullet, MuJoCo, ROS 2, Drake, OMPL, Pinocchio, MoveIt!, Habitat, Project Chrono, MRPT, YARP, and OpenRTM-aist.

The overall surface understandability impression scores range from 5 to 10, with a mean of 7.3. Gazebo (10), OMPL (9), and MoveIt! (9) score highest. GraspIt! (5) scores lowest.

5.10 Visibility and Transparency

Visibility and transparency assess the extent to which a project's development process and status are visible to external observers. Table 5.7 summarizes the key indicators.

Table 5.7: Visibility and transparency indicators.

Indicator	**Packages**	**%**
Development process defined (CI, contributing guide, etc.)	25	89%
Documents recording development process and status	28	100%
Development environment documented	28	100%
Release notes published	27	96%

Twenty-five of 28 packages have a defined development process, typically manifested through CI/CD workflows, contributing guidelines, pull request templates, and issue templates. All 28 packages use GitHub Issues or similar tools to record development status. All 28 document their development environment (build dependencies, toolchain requirements, etc.). Twenty-seven of 28 publish release notes.

Three packages are recorded as unclear for the defined-development-process metric: CoppeliaSim, ODE, and OMPL. For CoppeliaSim, the mixed licensing model means parts of the development process are not publicly visible. For ODE, the unclear result is consistent with its low activity level and sparse process documentation. For OMPL, the result is best read narrowly as the outcome of this measurement item, not as a claim that the project lacks public development infrastructure in every respect.

The overall visibility and transparency impression scores range from 5 to 10, with

a mean of 7.4. Gazebo (10), Drake (9), ROS 2 (9), OMPL (9), and MoveIt! (9) score highest. These packages publish thorough documentation of their development workflows. MORSE (5), Flightmare (5), Newton Dynamics (5), and GraspIt! (5) score lowest, consistent with their dead or low-activity status.

5.11 Repository Metrics

Table 5.8 presents key repository metrics for all 28 packages, including GitHub popularity indicators and codebase size measures. All data are from May 2026.

Notes: ROS 2 code line and comment counts are not directly comparable because the ROS 2 organization spans hundreds of repositories. ODE is hosted on BitBucket; star, fork and issue-closure metrics are reported as "—" because the BitBucket APIs used during measurement did not return values for these fields under our access configuration, not because they are conceptually unavailable on that platform. The "code lines" column reports SCC-measured code lines in the primary measured repository. † For PhysX, the commit count reflects only the public open-source SDK release on `NVIDIA-Omniverse/PhysX`; the broader internal NVIDIA codebase is not publicly accessible and is therefore not represented in this metric. ‡ Newton Dynamics is measured on the maintainer's current development repository, created in January 2026 after a repository migration; its star and fork counts exclude the community attention attached to the project's original repository (roughly 1,000 stars as of June 2026). The OROCOS row reflects the `orocos_kinematics_dynamics` (KDL) repository only, as noted in Section 5.1.

GitHub stars (a rough proxy for community attention) span almost three orders

Table 5.8: Repository metrics for the 28 selected packages (May 2026), listed alphabetically.

Software	Stars	Forks	Commits	Code Lines	% Comments	% Issues Closed
AirSim	18,159	4,889	3,562	134,331	10.3%	80.0%
Bullet/ PyBullet	14,465	3,074	9,405	1,528,236	2.8%	87.3%
CARLA	13,941	4,559	6,713	138,801	9.4%	82.0%
CoppeliaSim	146	46	1,206	292,460	5.1%	95.5%
DART	1,081	300	6,727	339,389	9.2%	100.0%
Drake	4,025	1,366	34,786	705,686	20.5%	90.0%
Flightmare	1,353	400	139	14,543	20.1%	34.6%
Gazebo	1,337	411	7,500	193,808	11.9%	56.7%
GraspIt!	213	86	733	110,614	2.1%	56.6%
Habitat	3,662	530	1,664	143,150	7.7%	75.7%
MORSE	367	156	4,356	141,241	2.7%	77.6%
MoveIt!	1,803	744	9,462	159,789	26.1%	80.5%
MRPT	2,133	658	10,231	504,916	9.4%	95.4%
MuJoCo	13,440	1,501	4,676	349,490	6.7%	91.7%
Newton Dynamics	24‡	5‡	17,755	1,585,430	15.7%	100.0%
ODE	—	—	2,119	147,055	18.8%	—
OMPL	2,047	690	5,341	105,532	27.4%	93.2%
OpenRAVE	801	355	10,375	623,306	19.2%	42.9%
OpenRTM-aist	24	14	4,360	86,965	26.3%	81.4%
OROCOS	879	443	1,229	18,063	28.0%	78.0%
PhysX (OSS)	4,532	614	40†	1,030,524	17.3%	67.9%
Pinocchio	3,354	537	13,081	185,472	11.9%	92.3%
Project Chrono	2,836	595	19,510	657,303	1.1%	96.8%
ROS 2	5,475	903	309	—	—	86.2%
Simbody	2,521	492	5,041	259,447	25.3%	59.5%
SOFA	1,194	352	27,404	349,961	4.5%	64.1%
Webots	4,345	2,025	15,746	581,442	1.9%	87.9%
YARP	590	214	19,020	843,982	15.1%	82.3%

of magnitude, from 24 (OpenRTM-aist, and Newton Dynamics with the repository-migration caveat noted under Table 5.8) to 18,159 (AirSim). The median among the 27 packages with GitHub star data is approximately 2,050. The correlation between stars and measurement scores is not straightforward. AirSim leads in stars by a wide margin but does not appear among the top-scoring packages in most quality categories. Conversely, Gazebo scores highly across most quality categories but has relatively modest star counts (1,337), likely because its community engagement predates the current GitHub repository organization and because its user base interacts with it primarily through ROS rather than directly through its GitHub page.

The number of commits ranges from 40 (PhysX, reflecting only the public open-source SDK release noted in Table 5.8) to 34,786 (Drake). If PhysX is excluded as a special public-SDK mirror of a broader internal codebase, the smallest measured commit count is 139 (Flightmare, which is now inactive). Commit activity in the last 12 months shows a similar range: Drake leads with sustained high monthly activity, while several dead packages (MORSE, GraspIt!, Flightmare) show zero or near-zero recent commits.

The percentage of code that is comments shows substantial variation, from 1.1% (Project Chrono) to 28.0% (OROCOS). The Domain X maintainability calculator uses 10% as a threshold; 15 of the 27 packages with comparable measurements (about 56%) exceed it. The packages below 10% include some of the largest codebases in the study (Bullet/PyBullet at 2.8%, SOFA at 4.5%, Project Chrono at 1.1%), so codebase size alone does not determine comment density.

5.12 Overall Observations

Table 5.9 summarizes the overall impression scores across all nine quality categories for each package, along with a simple unweighted per-package mean. Figure 5.1 plots the per-package mean as a ranked bar chart coloured by activity status, Figure 5.2 renders the full score matrix as a heatmap, and Figure 5.3 shows how the scores are distributed within each of the nine categories. The scores reported here are the raw 1–10 impression values assigned during measurement using the Domain X Appendix C calculator. The unweighted mean treats all nine qualities as equally important. A fully defensible AHP-weighted ranking would require a formal criterion-weighting elicitation, which this study does not perform. For illustration, Figure 5.4 reports an AHP ranking computed with the Domain X pairwise-comparison algorithm applied to these scores under equal category weights. Under equal weights the two orderings agree at the top; the figure illustrates how the Domain X weighting step operates, not a weighted verdict on the packages.

A handful of packages anchor the top of the table. Gazebo scores 10 in all nine categories and is the most consistently high-rated package in the study. A perfect sweep deserves comment. In every category the recorded evidence places Gazebo at the strongest level observed in this study: a one-step packaged installation with a documented validation procedure, tutorials whose outputs matched their documentation, CI-backed automated testing, an explicitly identified coding standard, contribution guidelines, and versioned release notes. The score means that, within this template and time budget, no measured indicator separated Gazebo from the top of the scale; it is not a claim that the software is flawless. Drake and MoveIt! score 9 or 10 across most categories, and ROS 2, OMPL, and Pinocchio sit just below them. These

Table 5.9: Overall impression scores by quality category (1–10 scale) with an unweighted per-package mean. I = Installability, C = Correctness, SR = Surface Reliability, SRo = Surface Robustness, SU = Surface Usability, M = Maintainability, Re = Reusability, SUnd = Surface Understandability, VT = Visibility/Transparency. The Mean column is the simple arithmetic mean of the nine category scores.

Software	**I**	**C**	**SR**	**SRo**	**SU**	**M**	**Re**	**SUnd**	**VT**	**Mean**
AirSim	8	8	8	8	8	8	8	7	7	7.8
Bullet/ PyBullet	9	7	8	7	7	7	8	7	7	7.4
CARLA	7	8	8	7	8	8	8	7	8	7.7
CoppeliaSim	8	8	8	7	7	7	8	7	7	7.4
DART	8	8	8	7	7	8	8	7	7	7.6
Drake	7	10	10	9	9	9	8	8	9	8.8
Flightmare	7	6	7	8	6	4	6	6	5	6.1
Gazebo	10	10	10	10	10	10	10	10	10	10.0
GraspIt!	5	5	6	5	5	4	5	5	5	5.0
Habitat	8	8	8	8	8	8	8	8	8	8.0
MORSE	9	8	8	8	8	8	5	6	5	7.2
MoveIt!	7	9	9	8	9	9	9	9	9	8.7
MRPT	8	8	7	8	7	8	8	8	8	7.8
MuJoCo	7	7	8	8	8	8	9	8	8	7.9
Newton Dynamics	6	5	6	6	5	5	6	6	5	5.6
ODE	4	6	7	7	6	6	7	7	7	6.3
OMPL	9	9	9	8	8	8	9	9	9	8.7
OpenRAVE	8	8	8	7	8	8	7	8	7	7.7
OpenRTM-aist	7	8	7	7	6	7	6	7	7	6.9
OROCOS	7	8	7	6	6	6	7	6	6	6.6
PhysX (OSS)	4	6	6	6	7	8	8	8	8	6.8
Pinocchio	10	9	9	8	8	9	9	8	8	8.7
Project Chrono	7	8	8	7	7	8	7	7	8	7.4
ROS 2	8	9	9	8	9	9	9	8	9	8.7
Simbody	7	7	7	7	7	7	7	7	7	7.0
SOFA	6	6	8	7	6	7	7	7	7	6.8
Webots	8	8	8	7	8	8	8	7	8	7.8
YARP	7	8	7	7	7	8	7	7	8	7.3

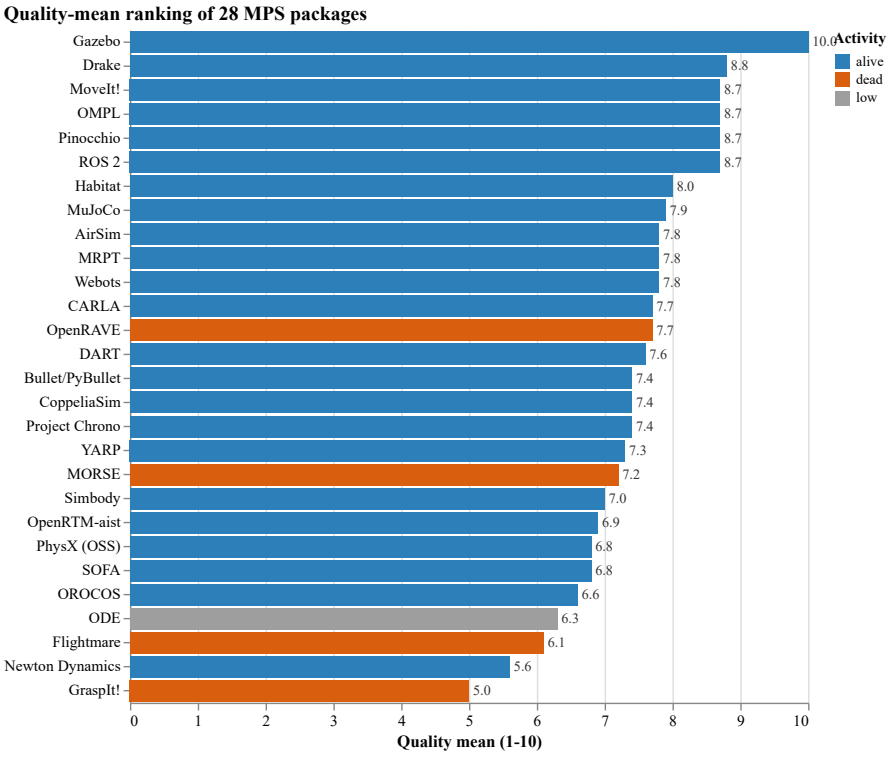

Figure 5.1: Quality-mean ranking of the 28 MPS packages (Table 5.9, Mean column), coloured by activity status (May 2026 snapshot). Bars give the unweighted mean of the nine category impression scores.

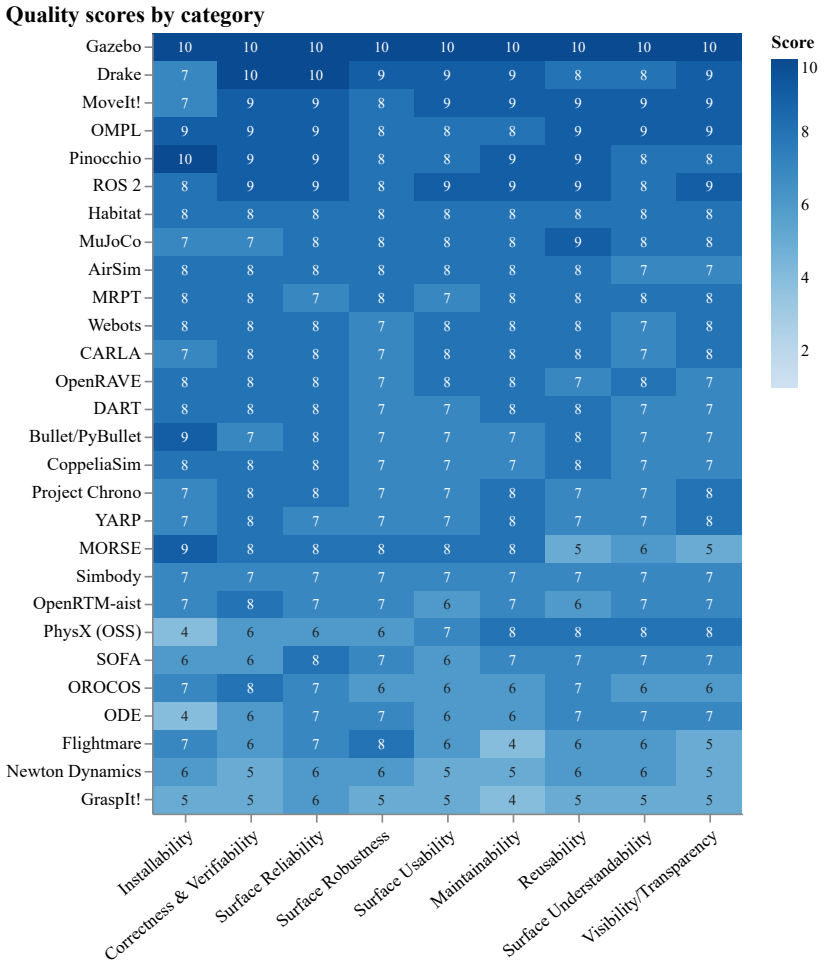

Figure 5.2: Quality impression scores (1–10) for the 28 MPS packages across the nine Domain X quality categories, rendered as a heatmap (Table 5.9). Packages are ordered top-to-bottom by their unweighted mean. Darker cells denote higher scores.

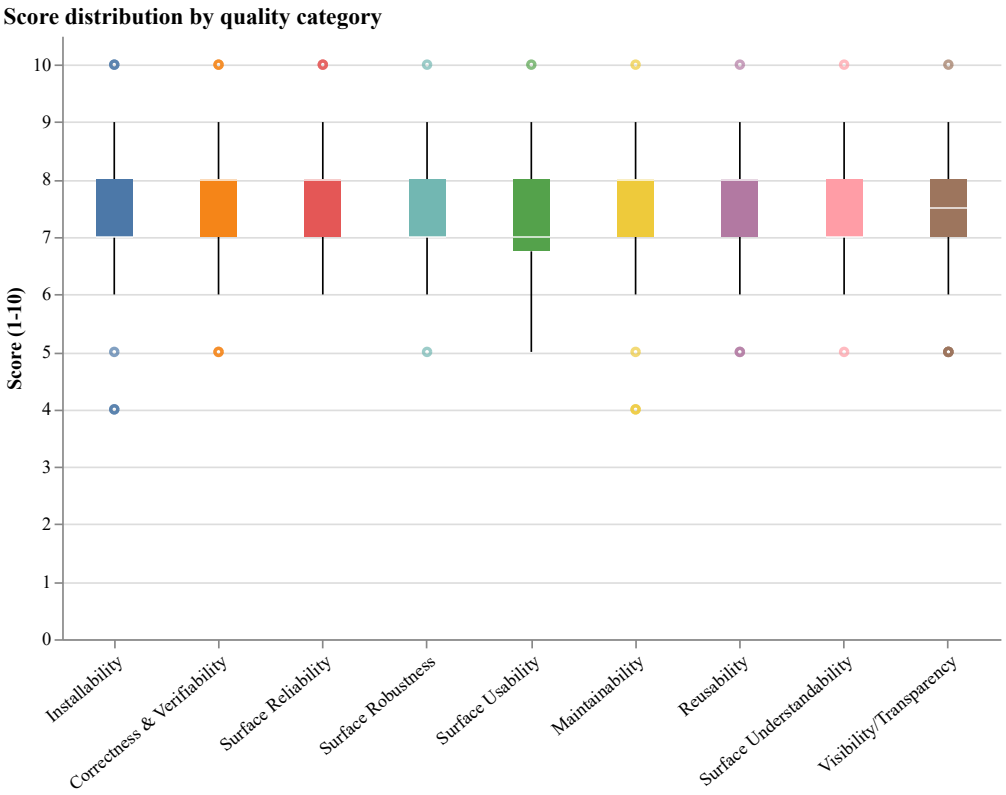

Figure 5.3: Distribution of the impression scores across the 28 packages within each of the nine quality categories (Table 5.9). Boxes span the interquartile range with the median marked; whiskers and points show the spread and outliers. The categories are bunched in a narrow 7–8 band, with Installability and Maintainability showing the widest low-end spread.

packages share several traits: established stewardship (large organizations or active research groups), continuous contributor activity, comprehensive documentation, automated testing wired into CI/CD, and published contribution guidelines and release notes. None of these traits is individually decisive, but the combination separates the top tier from the rest.

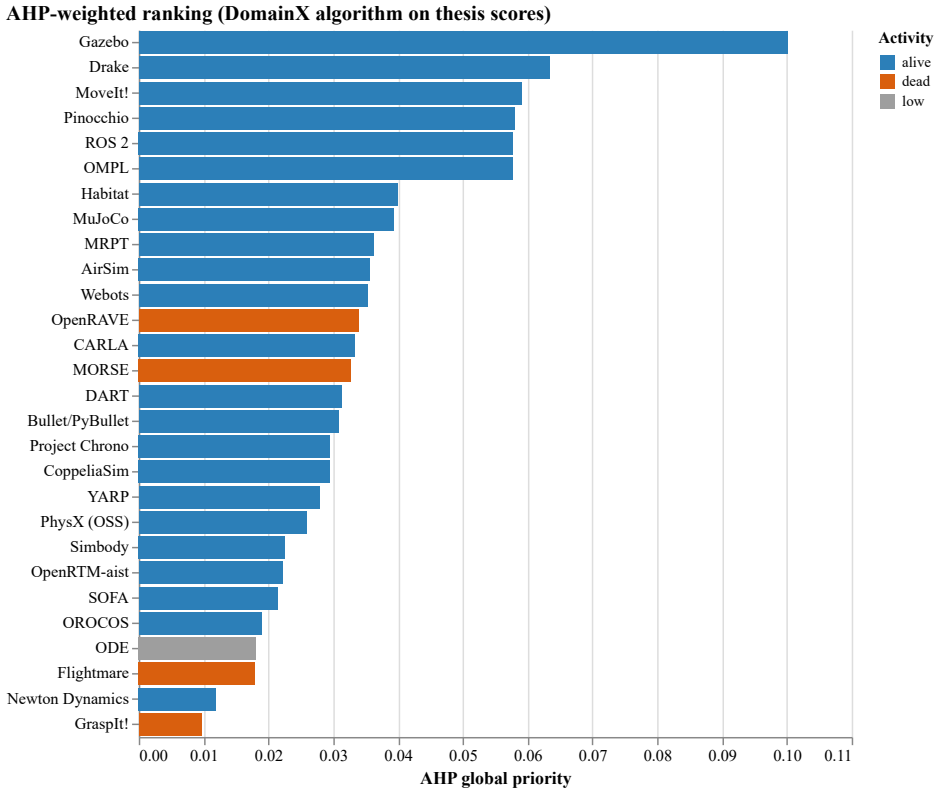

Figure 5.4: Illustrative AHP global-priority ranking obtained by applying the Domain X pairwise-comparison algorithm to the impression scores of Table 5.9 under equal category weights. Higher priority is better and the values sum to one across packages. Equal weights are used because no formal criterion-weighting elicitation was performed; the ordering reproduces the unweighted-mean leaders (Gazebo, Drake, MoveIt!, Pinocchio, ROS 2, OMPL) and is shown to illustrate the Domain X weighting step, not as an elicited result. Beyond the top tier, the AHP ordering and the unweighted-mean ordering differ at several mid-table positions, which is expected from the pairwise score mapping.

The bottom of the table tracks maintenance status, but only loosely. GraspIt! (last commit July 2020), Flightmare (last commit May 2023), and MORSE (last commit March 2022) appear in the lower tier for most quality categories. Newton Dynamics, though technically alive at the May 2026 measurement date, joins them, reflecting its largely individual-maintained codebase and thin documentation infrastructure. OpenRAVE is also classified as dead under the 18-month rule (last commit August 2024), but its quality scores cluster in the middle of the table rather than the bottom; the documentation, test infrastructure, and codebase organization built during years of active development have not visibly degraded in the period since. In short, packages that go inactive tend to accumulate outdated documentation, broken installation procedures, and unresolved issues, but the rate at which they do so depends on the engineering capital accumulated before they went quiet.

Installability is generally strong across the MPS domain. Section 5.2 reports broad instruction and automation coverage, with few installation breakages. The widespread adoption of standard package managers (APT, pip, Conda), containerization (Docker), and CMake-based builds in the robotics community is the most plausible explanation.

Documentation is universally present but varies in depth. All packages provide tutorials, user manuals, and API documentation. Only one package documents expected user characteristics, and only 14 of 28 explicitly identify a coding standard. The same gaps appear in the medical imaging and LBM samples (Smith et al., 2024a,b), so they look more like patterns of research software practice than peculiarities of MPS. Figure 5.5 summarizes this contrast across the template: the indicators tied to day-to-day installation and use sit at or near full coverage, while the two indicators that record

explicit engineering intent, coding standards and documented user characteristics, fall to 50% and 4%.

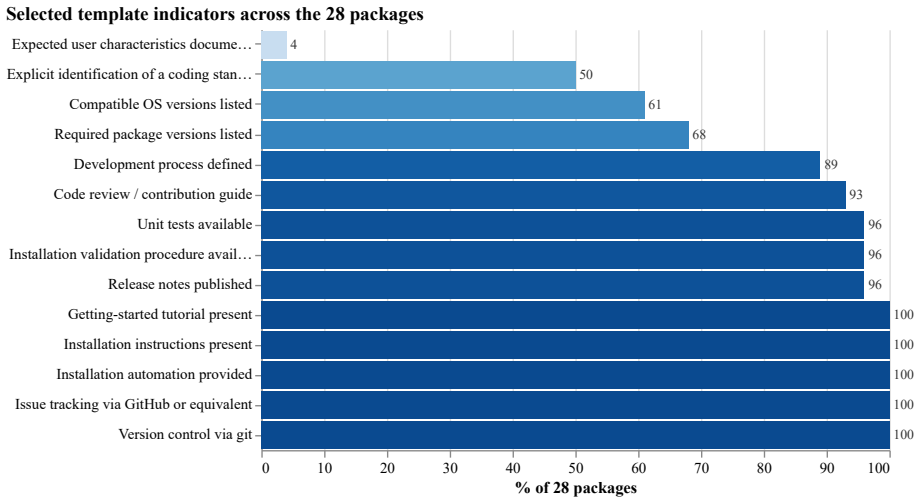

Figure 5.5: Coverage of selected measurement-template indicators across the 28 packages (May 2026), sorted from least to most common. The values restate Tables 5.2, 5.3, 5.5, 5.6, and 5.7, together with the documented-user-characteristics result of Section 5.6. Practical, user-facing artifacts cluster at or near 100%, while explicit coding standards (50%) and documented user characteristics (4%) are the clear outliers.

Repository activity is concentrated in a small subset of packages: Drake, SOFA, Project Chrono, YARP, and Newton Dynamics account for a disproportionate share of total commits, reflecting either sustained multi-year development by large teams or long-running individual-maintained histories. At the other extreme, several packages show minimal activity in the last 12 months.

The relationship between community popularity and measurement scores is not linear. Section 5.13 reports the explicit ranking comparison. Star counts accumulate over years and capture cumulative attention; the quality scores capture what each project's documentation, tests, and release infrastructure look like today, so the two indicators answer different questions and need not agree.

Chapter 6 interprets these results against prior State-of-the-Practice studies, and Chapter 10 answers the research questions in light of the evidence.

5.13 Comparison with Community Indicators

The Domain X quality mean reported in Section 5.12 reflects the per-package state of documentation, testing, release management, and related software engineering practices in May 2026. Community indicators such as GitHub stars, forks, and recent commit activity capture a different signal: cumulative visibility, user adoption, and word-of-mouth attention built up over time. The two need not agree. This section presents both rankings side by side, using the per-package quality mean from Table 5.9 and the star count from Table 5.8, and identifies where the indicators converge and where they diverge.

The 28 packages fall into four broad patterns, summarized visually in Figures 5.6 and 5.7.

Rough agreement at the top. Drake ($\Delta = +6$), ROS 2 ($\Delta = +2$), Pinocchio ($\Delta = +7$), and Habitat ($\Delta = +2$) appear in the top tier on both indicators; well-resourced projects with sustained development tend to combine strong engineering practice with substantial visibility.

High quality, low visibility. Gazebo ($\Delta = +16$), OMPL ($\Delta = +11$), and MoveIt! ($\Delta = +12$) score at or near the top on quality but sit well below their quality rank on stars. The Gazebo gap is partially explained at the end of Section 5.11: Gazebo's user base interacts with it primarily through ROS rather than its GitHub repository directly, so star counts undercount its reach. OMPL and MoveIt! are libraries embedded inside larger workflows and may attract attention as dependencies

Table 5.10: Quality-mean rank versus GitHub stars rank for the 28 selected packages. $\Delta = S - Q$: positive values indicate a higher quality rank than stars rank (under-recognized for its quality); negative values indicate a higher stars rank than quality rank (popular without proportional quality scores). Ties receive the same rank. ODE is hosted on BitBucket and is excluded from the stars rank, consistent with Table 5.8.

Software	**Q-Mean**	**Q-Rank**	**Stars**	**S-Rank**	**Δ**
Gazebo	10.0	1	1,337	17	+16
Drake	8.8	2	4,025	8	+6
MoveIt!	8.7	3	1,803	15	+12
OMPL	8.7	3	2,047	14	+11
Pinocchio	8.7	3	3,354	10	+7
ROS 2	8.7	3	5,475	5	+2
Habitat	8.0	7	3,662	9	+2
MuJoCo	7.9	8	13,440	4	−4
AirSim	7.8	9	18,159	1	−8
MRPT	7.8	9	2,133	13	+4
Webots	7.8	9	4,345	7	−2
CARLA	7.7	12	13,941	3	−9
OpenRAVE	7.7	12	801	21	+9
DART	7.6	14	1,081	19	+5
Bullet/ PyBullet	7.4	15	14,465	2	−13
CoppeliaSim	7.4	15	146	25	+10
Project Chrono	7.4	15	2,836	11	−4
YARP	7.3	18	590	22	+4
MORSE	7.2	19	367	23	+4
Simbody	7.0	20	2,521	12	−8
OpenRTM-aist	6.9	21	24	26	+5
PhysX (OSS)	6.8	22	4,532	6	−16
SOFA	6.8	22	1,194	18	−4
OROCOS	6.6	24	879	20	−4
ODE	6.3	25	—	—	—
Flightmare	6.1	26	1,353	16	−10
Newton Dynamics	5.6	27	24	26	−1
GraspIt!	5.0	28	213	24	−4

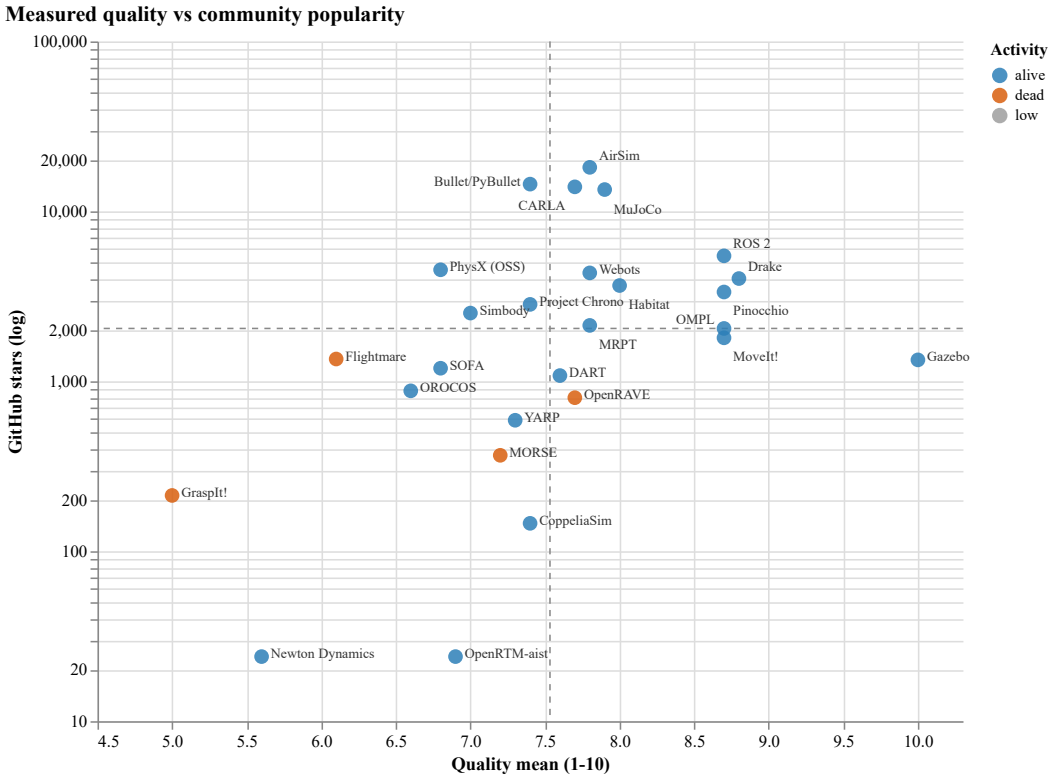

Figure 5.6: Measured quality mean (Table 5.9) versus GitHub stars (log scale, Table 5.8) for the 28 packages, coloured by activity status (May 2026 snapshot). Dashed lines mark the mean quality and the median star count, both computed over the 27 packages with star data. Packages in the lower-right combine high measured quality with comparatively modest popularity; those in the upper-left are popular without proportional quality scores.

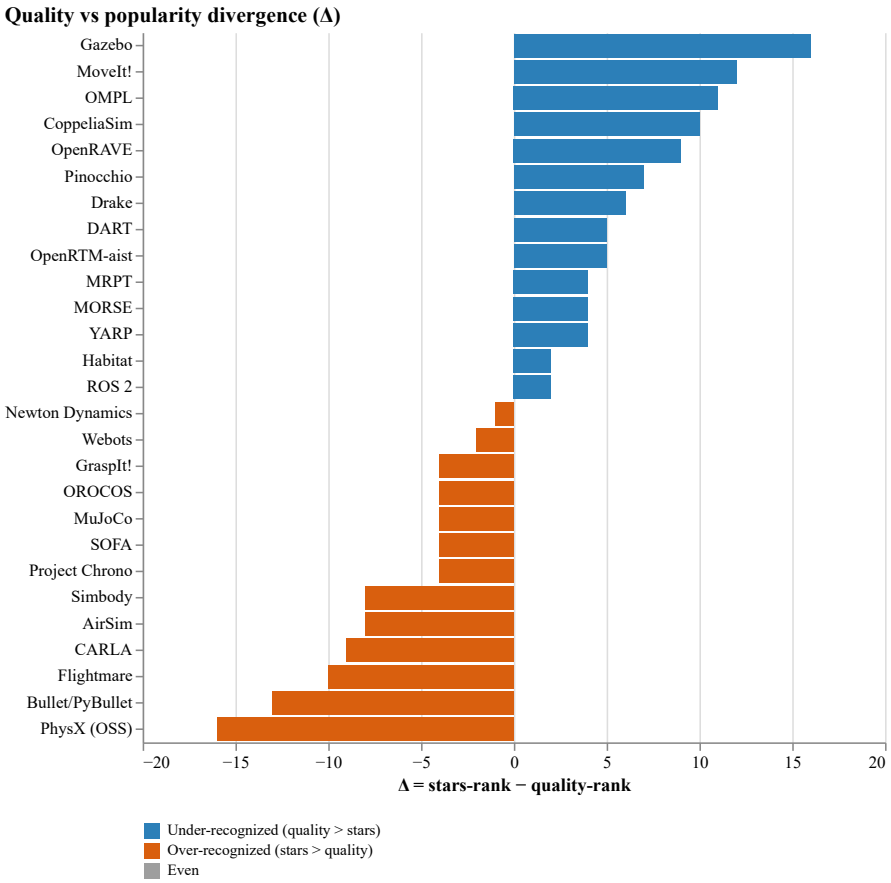

Figure 5.7: Rank divergence $\Delta = S - Q$ between the GitHub-stars rank and the quality-mean rank for each package (Table 5.10). Positive (blue) bars indicate packages that are under-recognized relative to their measured quality; negative (orange) bars indicate packages that are more popular than their quality scores alone would suggest.

rather than as standalone projects.

High visibility, mid-pack quality. AirSim ($\Delta = -8$), Bullet/PyBullet ($\Delta = -13$), CARLA ($\Delta = -9$), and PhysX (OSS) ($\Delta = -16$) sit at or near the top on stars but in the middle or lower tier on the quality mean. These projects share characteristics: broad user bases drawn from adjacent fields (gaming, autonomous-driving research, real-time physics), corporate or game-engine origins, and uneven research-software quality practice.

Consistently low. GraspIt! ($\Delta = -4$) and Newton Dynamics ($\Delta = -1$) cluster near the bottom on both indicators; inactive or narrowly used projects tend to accumulate gaps in documentation, releases, and visibility together. For Newton Dynamics this placement carries the repository-migration caveat from Section 5.1: measured against its original repository's accumulated stars, its popularity rank would sit closer to the middle of the table, although its quality rank would not change. OpenRTM-aist sits near the bottom on both indicators as well, but with $\Delta = +5$ its quality rank (21) is modestly better than its stars rank (26), so it belongs in this group only in the sense of low absolute standing on both axes, not as a case where popularity outruns quality.

These patterns are consistent with the finding from the Medical Imaging State-of-the-Practice study that GitHub popularity and measurement-based rankings do not align in a simple way (Smith et al., 2024a). They support the broader argument that community-facing indicators of visibility and adoption are not interchangeable with measured software engineering quality; treating stars as a quality proxy obscures meaningful differences across packages.

Chapter 6

Comparison with Other Research Software Domains

This chapter compares the findings for motion planning and simulation (MPS) software with prior State-of-the-Practice studies in other research software domains, especially Medical Imaging (MI) software (Smith et al., 2024a) and Lattice Boltzmann Method (LBM) software (Smith et al., 2024b). The goal is not to claim that one domain is better than another. The domains differ in age, user communities, computational goals, and measurement dates. Instead, the comparison identifies where MPS software follows patterns already observed in research software and where it appears to differ.

The comparison follows the structure used in the LBM study. That study compared LBM projects with other research software in terms of repository artifacts, tool usage, and development principles, processes, and methodologies (Smith et al., 2024b). The same three dimensions map directly to RQ3, RQ4, and RQ5 in this report.

6.1 Comparison of Repository Artifacts

RQ3 asks how MPS projects compare to other research software domains with respect to the artifacts present in their repositories. In the LBM study, repository artifacts were grouped by frequency and compared with research software development guidelines (Smith et al., 2024b). The same idea applies here: artifacts such as installation instructions, user manuals, API documentation, issue trackers, tests, release notes, and contribution guides make development activity visible and make software easier to install, inspect, reuse, and maintain.

Table 6.1 summarizes the main artifact observations from Chapter 5. These counts are based on the May 2026 measurement snapshot.

Table 6.1: Summary of selected artifacts and visible development evidence in MPS packages.

Artifact or evidence type	**Packages**	**%**
Public source repository	28/28	100%
Installation instructions	28/28	100%
Installation automation	28/28	100%
Getting-started tutorial	28/28	100%
User manual	28/28	100%
API documentation	28/28	100%
Issue tracking	28/28	100%
Version control	28/28	100%
Unit tests visible or available	27/28	96%
Release notes	27/28	96%
Contribution or code-review guidance	26/28	93%
Explicit coding standard	14/28	50%
Documented expected user characteristics	1/28	4%

The main artifact finding for MPS software is the near-universal presence of user-facing and developer-facing infrastructure. Every measured package provides installation instructions, some form of installation automation, a getting-started tutorial, a user manual, API documentation, issue tracking, and version control evidence. MPS compares well with prior State-of-the-Practice studies on this dimension. The MI study reported that 22 of 29 packages provided a user manual (Smith et al., 2024a), while all 28 MPS packages do so in the current measurement. The LBM study found

that installation guides and tutorials were common artifacts but API documentation was rare in the measured LBM sample (Smith et al., 2024b); in the MPS measurement, every package provides API documentation.

This difference likely reflects the way MPS software is used. Many MPS projects are not stand-alone research scripts; they are simulators, middleware systems, planning libraries, physics engines, or integration frameworks. Their users often need to install them into larger toolchains, call their APIs from other software, and combine them with robotics ecosystems such as ROS. That use pattern creates pressure for installation documentation, tutorials, and API references. The presence of these artifacts does not by itself prove high quality, but it does show that the MPS ecosystem has adopted many of the artifacts that make research software inspectable and reusable.

The comparison is less uniformly positive for formal software engineering artifacts. Only one MPS package documents expected user characteristics, and 14 of 28 explicitly identify a coding standard. The coding-standard result is stronger than in the MI and LBM comparison studies, but it still means that half of the measured MPS packages do not expose this artifact. The LBM study observed that research software projects often fall short of recommended artifacts such as roadmaps, coding standards, checklists, uninstall instructions, and formal design or requirements documentation (Smith et al., 2024b). The MPS results follow the same broad pattern: practical artifacts that help users install, run, and call the software are widespread, while artifacts that record design intent, expected user assumptions, and explicit development rules are less common.

The MPS artifact profile therefore appears stronger than MI and LBM in several

public-facing documentation categories, especially manuals and API documentation. Still, it does not eliminate the usual gap between research software practice and software engineering recommendations. MPS packages are generally well equipped for use and integration, but their repositories less often expose the rationale, process rules, and user assumptions behind that software.

6.2 Comparison of Tool Usage

RQ4 asks how MPS projects compare to other research software domains with respect to tool usage. In the LBM study, tool usage was discussed in terms of development tools, dependencies, and project management tools, with special attention to version control and continuous integration (Smith et al., 2024b). The current measurement provides strong evidence for version control, issue tracking, build or installation automation, testing, API documentation, and visible development infrastructure. Evidence for documentation-generation tools is narrower, since these tools are recorded for 12 of 28 packages.

Version control and issue tracking provide the most direct tool comparison. All 28 MPS packages use git-based version control and host public repositories on GitHub or an equivalent public platform. All 28 packages also use GitHub Issues or an equivalent issue tracker. The LBM study reported version control for 67% of all LBM packages and 73% of alive LBM packages (Smith et al., 2024b). The MPS result is higher, although part of that difference comes from the inclusion rule: repository-based measurement requires public repository evidence, so packages without suitable public repositories are filtered out before measurement. Even with that caveat, the result shows that the measured MPS ecosystem is strongly aligned with contemporary

repository-centered development.

Build and installation automation are also widespread in the MPS sample. All 28 packages provide some form of installation automation, such as build files, scripts, package-manager support, or container-based installation support. This matters because MPS packages often depend on compiled C++ code, Python bindings, physics libraries, graphics systems, middleware, and operating-system packages. Manual installation would make such systems difficult to reproduce. The strong installability result in Chapter 5 suggests that MPS maintainers have invested in build and installation tooling to reduce this burden.

Testing and verification tooling are also visible. Unit tests are available for 27 of 28 packages, automated testing is recorded for 26 packages, assertions are recorded for 23 packages, and documentation-generation tools are recorded for 12 packages. This matters for the LBM comparison because the LBM study treated test cases as an uncommon artifact and discussed automated testing as one of the prerequisites for effective continuous integration (Smith et al., 2024b). In the MPS sample, testing evidence is much more common. This does not mean that every package has complete test coverage, nor that the tests exercise realistic robotics workloads. It does show, however, that test infrastructure is a visible part of most measured MPS projects.

Continuous integration should be interpreted more cautiously. Chapter 5 records CI or an equivalent automated-testing setup for 26 of 28 packages and, for 25 of 28, visible development-process evidence such as CI/CD workflows, contribution guides, pull request templates, and issue templates. These counts do not mean that the packages all use continuous integration in the same way or to the same depth. Some may use CI for builds, others for tests, documentation, static checks, packaging, or

release automation. The narrower conclusion is that MPS repositories often expose automation and workflow infrastructure, but this report does not treat CI maturity as a separately ranked measure.

API documentation is another point of difference. The MPS measurement found API documentation for all 28 packages. This should be separated from tool usage: documentation-generation tools such as Doxygen or Sphinx are explicitly recorded for 12 MPS packages, not for the full set. The LBM paper reported that roughly half of the LBM packages mentioned documentation-generation tools and that API documentation was rare (Smith et al., 2024b). The MPS result is stronger on API documentation itself, likely because many MPS packages are libraries, frameworks, or middleware systems whose adoption depends on programmable interfaces.

Overall, MPS projects appear to use modern repository and development tools consistently in the categories visible from public repositories. The best-supported observations concern version control, issue tracking, installation automation, tests, and API documentation. Compared with MI and LBM, the clearest advantages are API documentation and testing evidence; basic repository infrastructure is at least comparable with MI and stronger than the LBM comparison point. The weaker point is not the absence of tools, but the uneven visibility of how tools are governed: coding standards, process rules, review expectations, and user assumptions are not documented as consistently as the tools themselves.

6.3 Comparison of Principles, Processes, and Methodologies

RQ5 asks how MPS projects compare to other research software domains with respect to principles, processes, and methodologies. This question is harder to answer from public artifacts alone because process is partly social. Repositories can show issue trackers, pull requests, release notes, contribution guides, and automated workflows, but they cannot fully reveal how teams make design decisions, how maintainers negotiate priorities, or how much informal review occurs outside the public repository.

Within that limitation, MPS projects show more visible process structure than many research software projects described in prior studies. Twenty-five of 28 MPS packages have evidence of a defined development process, typically through CI/CD workflows, contribution guides, pull request templates, issue templates, or similar repository artifacts. Twenty-seven of 28 publish release notes. Twenty-six of 28 provide contribution or code-review guidance. These artifacts indicate that many measured MPS projects expect external users or contributors and have created at least minimal structures for managing that interaction.

This contrasts with the LBM discussion of principles and process, where process evidence was often informal and was strengthened by interviews with developers (Smith et al., 2024b). LBM developers described small-team practices, lead developer roles, informal peer review, and issue tracking. The MPS interview component is planned separately in Chapter 7, but the current comparison is limited to public artifacts: MPS repositories frequently expose the artifacts associated with open collaboration, including issues, pull requests, contribution instructions, release

records, and workflow files.

The MPS process picture is still incomplete in two ways. First, 14 of 28 packages explicitly identify a coding standard, so this artifact is more common in MPS than in the comparison studies but still absent from half of the sample. Second, only one package documents expected user characteristics. This limits traceability between the intended audience and the design of tutorials, APIs, examples, and support channels. These gaps echo the LBM paper's observation that research software often has practical documentation but weaker formal requirements and design rationale (Smith et al., 2024b).

The heterogeneity of MPS software also affects process comparison. A full simulator such as Gazebo or Webots, a middleware platform such as ROS 2, and a planning library such as OMPL do not have identical development needs. Large ecosystem projects tend to expose more formal governance, release management, and contribution infrastructure. Smaller or older packages may provide source code and documentation but little explicit process description. For that reason, the comparison should not treat the 28 MPS packages as interchangeable products. The commonality analysis in Chapter 4 is important because it explains why different process expectations may be reasonable for different kinds of MPS software.

Taken together, the evidence suggests that MPS projects are relatively mature in visible, repository-based process infrastructure. They align with modern open-source practice through public issue tracking, version control, release notes, contribution guidance, and automation. They still resemble other research software domains in the weaker documentation of design rationale, user assumptions, coding standards, and formal process principles.

6.4 Summary of Cross-Domain Findings

The cross-domain comparison answers RQ3–RQ5 as follows. Table 6.2 first consolidates the quantitative comparison discussed across this chapter, placing the MPS counts alongside the corresponding figures reported by the medical imaging and Lattice Boltzmann method studies.

Table 6.2: Cross-domain comparison of repository artifacts, tools, and process evidence: motion planning and simulation (MPS, this study) against the medical imaging (MI) and Lattice Boltzmann method (LBM) State-of-the-Practice studies. MPS counts are out of 28 packages (Chapter 5, May 2026); MI counts are out of 29 packages (Smith et al., 2024a) and LBM counts out of 24 packages (Smith et al., 2024b). Percentages are taken over each study's own package count, so the columns are comparable despite the different denominators.

Artifact / evidence	**MPS (n=28)**	**MI (n=29)**	**LBM (n=24)**
Repository artifacts (RQ3)			
Version control	28 (100%)	29 (100%)	16 (67%)
Issue tracking	28 (100%)	28 (97%)	n/r
Tutorial (getting-started)	28 (100%)	18 (62%)	19 (79%)
User manual	28 (100%)	22 (76%)	13 (54%)
Installation instructions	28 (100%)	16 (55%)	n/r
API documentation	28 (100%)	7 (24%)	0 (0%)
Release notes	27 (96%)	22 (76%)	9 (38%)
Contribution / review guidance	26 (93%)	8 (28%)	n/r
Tools and process evidence (RQ4–RQ5)			
Unit tests available	27 (96%)	15 (52%)	2 (8%)
Continuous integration	26 (93%)	5 (17%)	3 (13%)
Documentation-generation tools	12 (43%)	n/r	13 (54%)
Explicit coding standard	14 (50%)	3 (10%)	3 (13%)
Documented user characteristics	1 (4%)	n/r	5 (21%)

Notes: MPS values restate Chapter 5. "n/r" marks an artifact that the cited study does not report as a count. The LBM API-documentation entry is zero because that study recorded no package with explicit API documentation (Smith et al., 2024b).

For RQ3, MPS projects compare favourably with prior research software domains

in the presence of repository artifacts. Installation instructions, installation automation, tutorials, user manuals, API documentation, issue tracking, version control, unit tests, release notes, and contribution guidance are common or near-universal in the measured MPS set. Compared with MI and LBM, the MPS sample appears especially strong in user manuals, API documentation, and testing evidence. Its basic repository infrastructure is comparable with MI and stronger than the LBM comparison point. The main gaps are formal artifacts: documented user characteristics, requirements-style information, design rationale, and explicit coding standards for the half of packages that do not record one.

For RQ4, MPS projects show strong tool adoption in the categories visible from public repositories. Version control and issue tracking are universal in the measured set, installation automation is universal, unit-test evidence is present in nearly all packages, and API documentation is present for every package. Read together, these numbers indicate that MPS software development is tightly connected to modern open-source tooling. The public, repository-measurable inclusion rule matters here: proprietary or closed-core robotics tools may follow different practices.

For RQ5, MPS projects show substantial public evidence of development processes, but the evidence is mostly artifact-based. Contribution guidance, release notes, issue tracking, and visible workflow infrastructure are common. More formal principles and process documentation are less consistent. This pattern places MPS software between two tendencies: it is more visibly tooled and documented than some prior research software domains, but it still shares the broader research software tendency to under-document design intent, requirements assumptions, and coding rules.

Taken as a whole, the measured MPS ecosystem shows a solid public-artifact profile. Its strengths are practical and integration-oriented: installability, tutorials, API documentation, version control, issue tracking, testing evidence, and release visibility. Its weaknesses are also familiar from prior research software studies: formal requirements, explicit user assumptions, coding standards, and design rationale remain less consistently documented. That mixed picture is the substance of this report's contribution: it characterizes the current state of practice in software development rather than recommending any particular software package.

Chapter 7

Developer Interview Component

This chapter records the planned role of the developer interview component in the study. The public-artifact measurements in Chapters 4 through 6 answer RQ1–RQ5. RQ6–RQ9 require evidence from developers about pain points, practices, and cross-domain lessons that are not fully visible in public repositories. At the current stage of the report, those interview findings have not yet been collected or analyzed, so this chapter does not present empirical answers to RQ6–RQ9. The sections follow the planned reporting flow rather than strict question order: the pain-point summary (RQ6) is followed directly by the practices that address those pain points (RQ8), because each practice will be reported against the pain point it targets, and the closing cross-domain comparison treats RQ7 and RQ9 together.

The interview protocol, recruitment logic, ethics status, and analysis procedure are described in Section 3.9 of Chapter 3. Once MacREM approval, recruitment, consent, and analysis are complete, this chapter will report only evidence that is supported by approved interview records or consented summaries.

7.1 Interview Evidence Base

This section will describe the evidence base for future interview findings: the number of contacted projects, the number of participating developers, the represented MPS software packages, the form of the records used for analysis, and any limits imposed by consent or confidentiality.

Until those data are available, no claims are made about developer-reported experience. The interview evidence is expected to complement the repository evidence used earlier in the report. Public artifacts show visible practices, such as documentation, issue tracking, releases, test evidence, and continuous-integration configuration. Interviews can provide less visible information, such as internal decision making, perceived development obstacles, informal review practices, funding or time constraints, and the reasons behind particular engineering choices.

7.2 Overview of Developer Pain Points

This section is the planned location for the answer to RQ6. It will summarize recurring pain points reported by MPS developers and will distinguish domain-wide themes from issues specific to one package or software role.

The grouping should be derived from interview evidence rather than imposed only from prior studies. Prior State-of-the-Practice reports suggest useful comparison categories, such as resource constraints, testing and correctness, documentation, maintainability, tooling, and project management, but the final categories for the MPS domain must reflect the interview responses.

7.3 Practices Used to Address Pain Points

This section is the planned location for RQ8: the specific best practices that MPS developers use, propose, or identify as relevant for addressing the pain points in RQ6 and the software qualities measured for RQ2. The analysis should separate practices that developers already use from practices they propose or are considering, because implemented practice and aspiration provide different kinds of evidence.

The analysis should connect each practice to the pain point it addresses. For example, if developers discuss practices related to documentation, testing, continuous integration, release management, contribution review, modularization, or user support, the chapter should state whether those practices were reported as current practice, past attempts, or future plans.

7.4 Analysis by Software Quality

This section will connect future interview findings to the software qualities measured in Chapter 5: installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency. The purpose is to explain why particular qualities are difficult to improve or how teams try to improve them in practice.

The analysis must keep developer-reported evidence separate from public-artifact evidence. For instance, an interview may reveal testing, review, release, or documentation practices that are not obvious from the repository. Conversely, a repository may expose artifacts that were not emphasized by the developer. These differences should refine the interpretation of the measurements, not replace the measurement

record.

7.5 Comparison with Other Research Software Domains

This section is the planned location for RQ7 and RQ9. It will compare future MPS interview findings with findings from prior State-of-the-Practice studies, particularly the medical imaging and Lattice Boltzmann method reports discussed in Chapter 6. The comparison should identify pain points that appear across multiple domains and pain points that seem more specific to MPS software.

The comparison will also support future-practice recommendations. A practice observed in another research software domain should be presented as a candidate practice for MPS only when the connection to an MPS pain point is clear. This avoids treating recommendations from other domains as automatically transferable.

7.6 Summary of Interview Evidence Needed

At the current stage, the interview component is a planned extension rather than a source of completed findings. The report therefore answers RQ1–RQ5 from public artifacts and cross-domain comparison, while RQ6–RQ9 remain dependent on approved interview recruitment, completed interviews, and systematic qualitative analysis.

Before this chapter can be converted into a findings chapter, the study needs a confirmed ethics status, a record of participant recruitment and consent, a coded set of developer responses, and a documented analysis linking pain points and practices

back to the research questions.

Chapter 8

Recommendations for Future Practice

The recommendations in this chapter come from the public-artifact measurements, the commonality analysis, and the comparison with prior State-of-the-Practice studies. They are meant for future MPS software practice; they are not a ranking of packages or a prescription that every project should adopt the same process. They are also necessarily partial answers to the research questions: RQ8 and RQ9 ask which practices address developer pain points, and the pain points themselves (RQ6) await the interview component described in Chapter 7. The recommendations below rest on the artifact evidence alone and should be revisited once interview evidence is available.

8.1 Recommendations for MPS Software Users

Users should choose packages by task fit before quality score. The selected set includes full simulators, physics engines, planning libraries, middleware, and robotics development frameworks. A highly ranked simulator may be the wrong choice for a project that needs a planning library, and a strong middleware framework cannot replace a physics engine. The commonality analysis in Chapter 4 should therefore be read alongside the measurement results.

Users should also treat public artifacts as evidence of adoption risk. Installation instructions, tutorials, API documentation, issue trackers, and release notes reduce the effort needed to evaluate and maintain a package. Sparse documentation, weak release history, or inactive repositories do not make a package unusable, but they increase the work needed before a project can depend on it. Popularity indicators deserve the same caution in the other direction: Chapter 5 shows that GitHub stars and measured engineering quality diverge for several packages, so a star count is not a substitute for checking the artifacts directly.

8.2 Recommendations for MPS Software Maintainers

Maintainers should make development practices visible. The measured MPS projects often provide installation guides, tutorials, API references, and issue trackers, but formal and explanatory artifacts are less consistent. Coding standards, user assumptions, design rationale, and requirements-style information are useful because many MPS packages are adopted by researchers, graduate students, industry developers,

and contributors who were not part of the original project.

Three gaps recur in the measurements and are inexpensive to close. First, only one of the 28 measured packages documents its expected user characteristics; a short statement of who the software is for would help users self-select and would give tutorial authors a defined audience. Second, half of the packages (14 of 28) do not explicitly identify a coding standard, even though all 28 exhibit consistent formatting in practice; naming the standard, or publishing the formatter configuration, turns an implicit convention into a checkable artifact for contributors. Third, compatible operating-system versions are listed by only 61% of the packages and required dependency versions by 68%; recording both would remove a common source of installation friction, which is where most of the low installability scores in Chapter 5 originate.

Maintainers should document the purpose of testing and automation rather than only exposing the workflow files. Continuous-integration evidence is common in the measured repositories, but CI can mean builds, tests, documentation generation, static checks, packaging, or release automation. A short note stating what the automation verifies, such as builds, unit tests, or packaging, would make the evidence easier for users and contributors to interpret.

Maintainers should also keep repository identity legible. Several measured packages distribute development across repositories or have migrated repositories; Chapter 5 documents PhysX, OROCOS, and Newton Dynamics as examples. A prominent pointer from old locations to the current development home keeps community signals such as stars, forks, and issue history interpretable for users and for studies like this one.

8.3 Recommendations for Future State-of-the-Practice Studies

Future State-of-the-Practice studies in heterogeneous software domains should separate domain membership from software role. MPS software is broad: packages may appear in the same workflow without serving the same function. Recording each package's role through commonality analysis helps avoid treating related tools as interchangeable products.

Future studies should also keep public-artifact evidence separate from developer-reported evidence. Repositories can reveal documentation, issue tracking, releases, testing evidence, and development visibility. They reveal less about internal decision making, informal review, funding constraints, and developer pain points. Those topics require interviews or other qualitative evidence, and claims about them should be tied to the interview records or consented summaries that support them.

Finally, future studies should record the observations behind each impression score at measurement time. The impression scores summarize more than the binary checklist answers (Section 3.7.1), and Chapter 9 notes the subjectivity this introduces. A short per-package note recording which observations drove each score would make the scores easier to audit and to reproduce, at little extra cost during measurement.

Chapter 9

Threats to Validity

Any study based on publicly available artifacts is constrained by what those artifacts can reveal. The threats below are organized by threat type: reliability, construct validity, internal validity, and external validity.

9.1 Reliability

All measurements are based on public evidence collected in May 2026 and reflect the state of each repository at that moment. The same project measured earlier or later might score differently. Some quality questions also require judgement, such as whether documentation is adequate or whether project metadata is complete enough to support a score. Where judgement calls were made, the rationale is recorded in the measurement template.

All measurements were performed by a single assessor without an independent inter-rater check, so individual judgement calls are not cross-validated. The impression scores additionally summarize observations beyond the recorded checklist

answers (Section 3.7.1), which introduces subjectivity that the recorded data alone cannot fully reconstruct. The time budget of one to four hours per package also bounds the depth of every check; the surface measures are shallow by design.

9.2 Construct Validity

The measurements use public artifacts as evidence for software qualities. This is a practical choice, but it does not capture every aspect of the underlying constructs. For example, visible testing evidence does not prove that a package is correct, and visible issue-tracking activity does not fully describe maintainability. Source appearance counts also show how often a package appears across candidate sources, not how good the software is.

9.3 Internal Validity

The candidate set was built from academic survey papers, community-curated lists, and LLM-assisted discovery. LLM-generated lists supplemented the academic and community-curated sources but did not drive them. These lists reflect the training data of the model rather than an independent survey of the MPS domain, so a package unknown to the model could be absent from its output. Filtering decisions also affect the measured set: a package can be excluded because it is outside scope, unavailable as open source, inactive, or not measurable from public repositories.

9.4 External Validity

The selected set covers open-source MPS software that can be measured through public artifacts. Proprietary platforms, discontinued projects, and narrowly scoped libraries may differ from the measured packages. The results therefore describe the selected open-source MPS sample, not all robotics software. Because the packages play different roles, comparisons should be interpreted as evidence about development practice across a domain rather than as direct product-to-product comparisons.

The installation-based measurements were performed on a Linux virtual machine, so the installability, surface reliability, and surface robustness results reflect Linux behaviour only. The same packages may install and behave differently on Windows or macOS, and installation routes available at measurement time, such as distribution packages, may not exist for other operating systems or for later distribution releases.

Chapter 10

Conclusions

10.1 Summary

This report assessed the state of the practice for robotics motion planning and simulation (MPS) software by adapting the Domain X methodology to a new domain. Candidate packages were discovered from academic surveys, community-curated lists, and auxiliary LLM-assisted discovery, consolidated into 295 unique candidate names, ranked by mention count, and filtered by open-source availability, repository-based measurability, and maintenance activity. The resulting 28 packages, spanning full simulators, physics and dynamics engines, planning libraries, manipulation frameworks, and middleware, were measured in May 2026 against the Domain X template: roughly 108 fields per package covering project metadata, nine quality categories, and repository metrics. A commonality analysis described the selected packages as a software family, and the findings were compared with prior State-of-the-Practice studies of medical imaging and Lattice Boltzmann method software. A developer-interview component was designed but, pending ethics approval and recruitment, has not yet

been executed.

10.2 Answers to Research Questions

RQ1 (candidate packages). Chapter 3 and Appendix A answer RQ1. Consolidating the three discovery sources produced 295 unique candidate names. Filtering by open-source availability, repository-based measurability, activity status, and scope coherence reduced the leading candidates to the 28 measured packages; notable exclusions include MATLAB/Simulink, Unity, NVIDIA Isaac Sim, and RaiSim (closed core components), MRDS and USARSim (inactive), and GPMP2 and CHOMP (single-algorithm scope).

RQ2 (quality ranking). Chapter 5 answers RQ2. Across the nine quality categories, the unweighted quality mean is led by Gazebo (10.0), Drake (8.8), and a tied group of MoveIt!, OMPL, Pinocchio, and ROS 2 (8.7); GraspIt! (5.0), Newton Dynamics (5.6), and Flightmare (6.1) rank lowest, consistent with their inactive or thinly maintained status. Category means are tightly bunched (7.3–7.8), so the rankings separate a strong top tier and a weak bottom tier more clearly than they separate the middle of the table. An elicited-weight AHP ranking remains future work; an illustrative equal-weight computation reproduces the same leaders.

RQ3 (repository artifacts). Chapter 6 answers RQ3. Practical artifacts are near-universal in the MPS sample: installation instructions, installation automation, tutorials, user manuals, API documentation, issue tracking, and version control are present for all 28 packages, which compares favourably with the medical imaging and LBM samples. The recurring gaps are formal artifacts: documented user characteristics (1 of 28) and explicit coding standards (14 of 28).

RQ4 (tool usage). The measured MPS projects are tightly connected to modern open-source tooling: git-based version control and issue tracking are universal, automated testing or CI evidence appears for 26 of 28 packages, and documentation-generation tools are recorded for 12. Part of this uniformity is a selection effect of the repository-based inclusion rule, but within the measured set, tool adoption is stronger than in the LBM comparison point and at least comparable with medical imaging.

RQ5 (principles, processes, and methodologies). Process evidence is visible but artifact-bound: 25 of 28 packages expose a defined development process, 27 publish release notes, and 26 provide contribution guidance. What repositories do not reveal, design rationale, requirements assumptions, and internal decision making, remains under-documented in MPS software, the same broad pattern reported for other research software domains.

RQ6–RQ9 (pain points and practices). These questions require developer-interview evidence and remain open. Chapter 7 records the planned evidence base and reporting structure, and Chapter 8 gives artifact-based recommendations that will be revisited once interview findings are available.

10.3 Key Findings

Four findings stand out from the artifact evidence. First, the practical engineering infrastructure of MPS software is strong: every measured package can be installed from documented instructions with some form of automation, and only two installations broke during measurement, both recoverably. Second, the weaknesses are concentrated in formal and explanatory artifacts rather than in tooling: user assumptions, coding standards, and design rationale are the recurring gaps. Third, community

popularity and measured quality are different signals: Gazebo scores highest on quality while sitting in the middle of the star ranking, and several highly starred packages place mid-table on quality, so popularity indicators should not be used as quality proxies. Fourth, activity status only loosely predicts measured quality: packages that go inactive decay at different rates depending on the engineering capital they accumulated while active, as the contrast between OpenRAVE and GraspIt! illustrates.

10.4 Future Work

The most immediate direction is the developer-interview component: completing ethics approval, recruitment, and analysis would answer RQ6–RQ9 and connect the measured artifact gaps to the pain points developers actually experience. A second direction is an elicited-weight AHP ranking with sensitivity analysis, which would turn the per-quality measurements into a defensible weighted ordering. Longitudinal repeat measurements would track how the domain's practices change over time, and the same methodology could be extended to adjacent robotics software domains.

One further direction concerns the measurement process itself. The template used in this study contains roughly one hundred fields per package (Appendix C), and completing it by hand is the most time-consuming part of the work. As an exploratory check, an AI coding agent was asked to install, verify, and measure two of the selected physics engines on its own, using only the official documentation and the measurement template. The agent installed and verified both packages and filled the template; its factual entries were tied to public sources, while its subjective scores still needed human review. The trial is documented in Appendix F. With a person checking the evidence, this kind of agent-assisted measurement could make it practical

to cover more packages, or to repeat the measurements over time, without straining the time budget that constrains a manual study.

Appendix A

Full Candidate Software List

This appendix records the leading candidate software packages considered during candidate discovery (Section 3.3). Consolidating the names collected from academic survey papers, community-curated lists, and LLM-assisted discovery produced 295 unique candidates; the full ranked list is preserved in the supplementary project materials, and the counted sources are listed in Appendix B. Table A.1 records every candidate that received at least four mentions, together with the explicitly excluded tools and a representative sample of lower-count candidates.

Each entry shows the candidate's primary role in the MPS workflow, its consolidated mention count, and its status after filtering. The status values are defined as follows. *Measured*: selected for measurement and reported in Chapter 5. *Excluded*: removed by an explicit, documented filtering decision (open-source availability, activity, or scope), with the rationale for each exclusion given in Appendix D. *Not selected*: remained in the candidate pool but was not chosen, because it fell below the mention-count cutoff, failed one of the filtering criteria of Section 3.5, or is subsumed by a measured package. In the Mentions column, "w/ X" means the candidate is counted

within the consolidated entry of measured package X, and "—" means the candidate has no entry in the consolidated mention-count list and entered the candidate discussion through the scope and filtering review rather than through the counted sources.

Table A.1: Candidate software packages considered for measurement, with consolidated mention counts.

Package	**Primary role**	**Mentions**	**Status**
AirSim	Simulator (autonomous vehicles)	7	Measured
Bullet/PyBullet	Physics engine	16	Measured
CARLA	Simulator (autonomous driving)	6	Measured
CoppeliaSim (V-REP)	Simulator (general robotics)	13	Measured
DART	Dynamics library	6	Measured
Drake	Toolbox (dynamics, planning, control)	6	Measured
Flightmare	Simulator (quadrotor)	4	Measured
Gazebo	Simulator (general robotics)	22	Measured
GraspIt!	Manipulation (grasping)	4	Measured
Habitat	Simulator (embodied AI)	4	Measured
MORSE	Simulator (academic robotics)	6	Measured
MoveIt!	Manipulation (motion planning)	4	Measured
MRPT	Library (mobile robotics)	4	Measured
MuJoCo	Physics engine	12	Measured
Newton Dynamics	Physics engine	4	Measured
ODE (Open Dynamics Engine)	Physics engine	10	Measured
OMPL	Motion planning library	5	Measured
OpenRAVE	Planning framework	9	Measured
OpenRTM-aist	Middleware (RT components)	4	Measured

Package	Primary role	Mentions	Status
OROCOS	Control framework	5	Measured
PhysX (open-source SDK)	Physics engine	8	Measured
Pinocchio	Dynamics library	5	Measured
Project Chrono	Multi-physics simulation	4	Measured
ROS 2	Middleware and SDK	10	Measured
Simbody	Multibody dynamics library	5	Measured
SOFA	Simulation framework	5	Measured
Webots	Simulator (general robotics)	16	Measured
YARP	Middleware (humanoid robotics)	4	Measured
NVIDIA Isaac Sim	Simulator (high-fidelity, AI)	9	Excluded (non-OSS)
Unity	Game engine / simulator	8	Excluded (non-OSS)
MATLAB / Simulink	Robotics development environment	6	Excluded (non-OSS)
RaiSim	Physics simulator	4	Excluded (non-OSS)
MRDS	Simulator (Microsoft)	5	Excluded (inactive)
USARSim	Simulator (robot/automation)	5	Excluded (inactive)
GPMP2	Motion planning algorithm (reference impl.)	—	Excluded (scope)
CHOMP	Motion planning algorithm (reference impl.)	1	Excluded (scope)
Player / Stage	Middleware and 2D/3D simulation	6	Not selected
ROS (ROS 1)	Middleware (predecessor of ROS 2)	w/ ROS 2	Not selected
iGibson / OmniGibson	Simulator (interactive household)	3	Not selected
ManiSkill	Simulator (manipulation benchmark)	3	Not selected

Package	Primary role	Mentions	Status
SAPIEN	Simulator (general robotics)	3	Not selected
RBDL	Rigid body dynamics library	3	Not selected
Crocoddyl	Optimal control library (DDP)	3	Not selected
RobotStudio	Industrial robot simulator (ABB)	3	Not selected
robosuite	Simulator (manipulation, MuJoCo-based)	w/ MuJoCo	Not selected
Orocos KDL	Kinematics and dynamics (component of OROCOS)	w/ OROCOS	Not selected
TrajOpt	Trajectory optimization library	2	Not selected
SBPL	Motion planning library (search-based)	2	Not selected
KineoWorks	Path planning (Siemens, commercial)	2	Not selected
Robotics Toolbox for MATLAB	Robotics toolbox (MATLAB)	2	Not selected
FCL	Collision and proximity queries	2	Not selected
Crazyswarm / Crazyswarm2	Multi-drone control and planning	2	Not selected
CasADi	AD and nonlinear optimization	2	Not selected
RoboDK	Industrial robot simulator and OLP	2	Not selected
Tesseract	Planning environment (optimization)	1	Not selected
iDynTree	Multibody dynamics, estimation	1	Not selected
STOMP	Motion planning algorithm	1	Not selected
Navigation2 (Nav2)	Mobile robot navigation stack	1	Not selected
Polymetis	Robot controllers (PyTorch)	1	Not selected

Package	Primary role	Mentions	Status
acados	Real-time NMPC and MHE solvers	1	Not selected
do-mpc	Python MPC and MHE	1	Not selected
Delmia	Digital manufacturing suite (Dassault)	1	Not selected
ros2_control	Robot control framework (ROS 2)	—	Not selected

The 28 measured packages were selected after the mention-count ranking and the filtering criteria of Section 3.5. Among the not-selected entries, proprietary or industrial tools such as *KineoWorks*, *Robotics Toolbox for MATLAB*, *RoboDK*, *RobotStudio*, and *Delmia* fall outside the open-source criterion; most others fell below the mention-count cutoff, and *Player / Stage*, despite six mentions, no longer meets the 18-month activity criterion. *ROS* (ROS 1) is consolidated with ROS 2 in the mention counting; the active successor was measured so that the results reflect current practice.

Appendix B

Mention Count Source List

This appendix lists the sources used to construct the consolidated mention-count ranking described in Section 3.4. Sources fall into three groups: community-curated robotics software lists, academic survey or comparison papers, and auxiliary LLM-assisted discovery lists. The LLM-assisted lists were used only as supplementary discovery aids, as discussed in Section 3.3.3.

Community-Curated Sources

Table B.1 lists the 15 community-curated sources used for candidate discovery. Each source is a public list, GitHub Topic page, or Wikipedia article that aggregates robotics software relevant to motion planning or simulation. The four GitHub Topic pages are retained because they are recorded in the source catalogue in the supplementary project materials. They are dynamic GitHub aggregations, so copied page text was not preserved in the raw-material archive in the same way as the static curated-list pages.

Table B.1: Community-curated sources used for mention counting.

Source	**Description**
GitHub Topic: motion-planning	GitHub Topic page aggregating active motion planning projects.
GitHub Topic: robotics-simulation	GitHub Topic page aggregating robotics simulation projects.
GitHub Topic: path-planning	GitHub Topic page for path and motion planning.
GitHub Topic: robotics-navigation	GitHub Topic page for navigation and planning.
best-of-robot-simulators	Curated list of robotic simulators (knmcguire).
Awesome Robotics Libraries	Robotics library list with planning and simulation sections (jslee02).
Awesome Robotics (ahundt)	Comprehensive robotics list covering planning, optimization, and simulation.
Awesome Robotics (kiloreux)	Community-maintained robotics list with planning and simulation entries.
Awesome Motion Planning	Motion planning list covering classical and modern libraries (AGV-IIT-KGP).
Awesome Multibody Dynamics Simulation	Multibody dynamics and simulation list (jslee02).
Awesome Robotic Tooling	Robotic tools and libraries list (Ly0n).
Awesome ROS 2	ROS 2 ecosystem list including navigation, planning, and simulation (fkromer).
Awesome Webots	Webots-focused ecosystem list (lukicdarkoo).
Awesome Robotics Projects	Robotics projects list with planning and simulation components (mjyc).
Wikipedia: Robotics simulator	Wikipedia article aggregating robotics simulator entries.

Academic Sources

Table B.2 lists the ten academic survey and comparison papers used for candidate discovery, with full bibliographic entries in the reference list. These papers survey or compare simulators, physics engines, and development frameworks used in robotics research; the second column gives representative packages from the selected set that each paper discusses.

LLM-Assisted Discovery Sources

Table B.3 lists the auxiliary LLM-assisted discovery outputs archived with the source catalogue. These outputs were not treated as authoritative sources; they were used to surface additional candidate names that were then consolidated, filtered, and checked against public evidence before any package was selected for measurement.

The source catalogue with community-source URLs and archived LLM-assisted outputs, the raw captured text from the static public web lists, and the consolidated ranking with per-package mention counts are all preserved in the supplementary project materials that accompany this report.

Table B.2: Academic survey and comparison papers used for mention counting.

Academic source	**Example packages discussed**
Brugali et al. (2007), "Trends in Robot Software Domain Engineering" (book chapter)	ROS, OROCOS, OpenRTM-aist, Gazebo, YARP
Žlajpah (2008), "Simulation in Robotics"	GraspIt!, ODE, Newton Dynamics, CoppeliaSim
Reckhaus et al. (2010), "An Overview about Simulation and Emulation in Robotics"	GraspIt!, OpenRAVE, Gazebo
Harris and Conrad (2011), "Survey of Popular Robotics Simulators, Frameworks, and Toolkits"	Gazebo, ROS, OpenRAVE, MRPT, Webots
Staranowicz and Mariottini (2011), "A Survey and Comparison of Commercial and Open-Source Robotic Simulator Software"	Webots, Gazebo
Iñigo-Blasco et al. (2012), "Robotics Software Frameworks for Multi-Agent Robotic Systems Development"	ROS, OROCOS, OpenRTM-aist, Gazebo, YARP
Collins et al. (2021), "A Review of Physics Simulators for Robotic Applications"	Gazebo, CoppeliaSim, Webots, MuJoCo, CARLA, SOFA, Project Chrono
Körber et al. (2021), "Comparing Popular Simulation Environments in the Scope of Robotics and Reinforcement Learning"	Gazebo, MuJoCo, PyBullet, Webots, DART, Simbody, PhysX
Farley et al. (2022), "How to Pick a Mobile Robot Simulator"	Gazebo, Webots, MORSE, CoppeliaSim, ODE, DART, Simbody
Kargar et al. (2024), "Emerging Trends in Realistic Robotic Simulations"	Gazebo, Webots, CoppeliaSim, MuJoCo, MORSE, PhysX, ODE, DART

Table B.3: LLM-assisted discovery sources archived with the source catalogue.

Source	**Role in candidate discovery**
GPT-generated robotics software list	Auxiliary broad-discovery list of open-source robotics software, simulators, planning libraries, physics engines, and related tooling.
Gemini-generated robotics software table	Auxiliary consolidated table of robotics frameworks, motion-planning tools, physics engines, and simulation platforms.

Appendix C

Measurement Template

This appendix reproduces the structure of the measurement template used in this report. The template is adapted from the template in Appendix B of Smith et al. (2021) and is grouped into project metadata, the nine quality categories described in Section 2.4, and repository metrics collected in the GitHub-style, GitStats-style, and SCC-style groups of Section 3.7.2. The full per-package responses are recorded in the measurement spreadsheet included in the supplementary project materials, with one row per question and one column per package. Each quality section ends with an overall impression score on a 1–10 scale.

Project Metadata

- Software name.
- URL.
- Affiliation: institution(s).

- Software purpose.
- Number of developers (commits-based count from repository logs).
- Funding source (unfunded, unclear, or funded with a string identifying the source).
- Initial release date.
- Last commit date.
- Status (alive, dead, or unclear; alive requires commits in the previous 18 months).
- License (GNU GPL, BSD, MIT, terms of use, trial, none, unclear, other).
- Supported platforms (Windows, Linux, macOS, Android, other).
- Software category (concept, public, or private).
- Development model (open source, freeware, commercial, or unclear).
- Number of publications mentioning the software.
- Source code URL.
- Programming languages.
- Whether performance was considered.

Installability

- Are installation instructions present?

- Are the instructions in one place (single document or web page)?
- Are the instructions linear (a single sequence of steps)?
- Are the instructions written assuming no dependencies are pre-installed?
- Are compatible operating system versions listed?
- Is the installation automated (makefile, script, installer, container)?
- If installation broke, was a descriptive error message displayed?
- Is there a specified way to validate the installation?
- Number of installation steps (including manual steps such as unzip).
- Operating system used for installation.
- Number of extra packages required before or during installation.
- Are required package versions listed?
- Are dependency installation instructions provided?
- Were there problems caused by uninstall (where applicable)?
- Overall installability impression (1–10).

Correctness and Verifiability

- Is there any reference to requirements specifications or theory manuals?

- What tools or techniques are used to build confidence in correctness (literate programming, automated testing, assertions, continuous integration, etc.)?
- Is there a getting-started tutorial?
- Are tutorial instructions linear?
- Does the tutorial provide an expected output?
- Does the tutorial output match the expected output?
- Are unit tests available?
- Is there evidence that performance was considered?
- Overall correctness and verifiability impression (1–10).

Surface Reliability

- Did the software break during installation?
- If installation broke, was a descriptive error message displayed?
- Did the software break during the initial tutorial test?
- If the tutorial test broke, was a descriptive error message displayed?
- If the tutorial test broke, was recovery possible?
- Overall surface reliability impression (1–10).

Surface Robustness

- Does the software handle unexpected or unanticipated input gracefully (data of the wrong type, malformed input, etc.)?
- For text input files, does the software handle modified line endings or other text-file edge cases?
- Overall surface robustness impression (1–10).

Surface Usability

- Is there a getting-started tutorial?
- Is there a user manual?
- Are expected user characteristics documented?
- What is the user support model (FAQ, user forum, email address for questions, etc.)?
- Overall surface usability impression (1–10).

Maintainability

- What is the current version number?
- Is there information on code review or contribution practices?
- Are non-code artifacts available, and which kinds?

- What issue tracking tool is in use?
- What percentage of identified issues are closed?
- What percentage of code consists of comments?
- Which version control system is in use?
- Overall maintainability impression (1–10).

Reusability

- How many code files are there?
- Is the API documented?
- Overall reusability impression (1–10).

Surface Understandability

- Is indentation and formatting style consistent?
- Is a coding standard explicitly identified?
- Are code identifiers consistent and meaningful?
- Are constants other than 0 and 1 hard-coded into the program?
- Are comments clear?
- Are parameters in the same order for all functions?

- Is the name or URL of any algorithm used mentioned?
- Is the code modularized?
- Overall surface understandability impression (1–10).

Visibility and Transparency

- Is the development process defined?
- Are there documents recording the development process and status?
- Is the development environment documented?
- Are release notes available?
- Overall visibility and transparency impression (1–10).

Repository Metrics

Counts collected from the primary repository at the measurement date. For projects with multiple repositories, the choice of the primary repository is recorded as part of the measurement notes.

- Number of text-based files.
- Number of binary files.
- Total lines in text-based files.
- Total lines added to text-based files.

- Total lines deleted from text-based files.
- Total number of commits.
- Commits per year for the last five years (or since project start if younger).
- Commits per month for the last twelve months.
- Code lines, comment lines, and blank lines in text-based files.
- Number of stars.
- Number of forks.
- Number of repository watchers.
- Number of open pull requests.
- Number of closed pull requests.
- Number of open issues.
- Number of closed issues.

Each substantive question above corresponds to one row in the measurement spreadsheet, with the per-package answer recorded in the column for that package. Including the supplementary entries (overall impression scores per category and the free-text additional-comments fields), the template contains approximately 108 fields per package.

Appendix D

Excluded Software Packages

This appendix records software packages that were considered during candidate discovery but excluded before measurement. Exclusions follow the criteria described in Section 3.5: open-source availability, repository-based measurability, and activity and maintenance status. A small number of additional packages were excluded to keep the selected set methodologically coherent, also described below.

Excluded for Lack of Open-Source Availability

The packages in Table D.1 have proprietary core components or closed source code, which prevents the repository-based measurement described in Section 3.7. Dr. Giamou confirmed that excluding these tools on methodological grounds does not misrepresent the MPS domain.

Table D.1: Packages excluded for lack of open-source availability.

Package	**Reason for exclusion**
MATLAB / Simulink	Proprietary software; source code not publicly available; development process not accessible through public repositories.
Unity	Game engine with robotics simulation capabilities, but the core simulation engine is proprietary.
NVIDIA Isaac Sim	Built on NVIDIA Omniverse; core platform components are not open source.
RaiSim	High-performance physics simulator; source code not publicly available under an open-source license.

Excluded for Inactivity

The packages in Table D.2 were excluded under the Domain X alive/dead rule (Section 3.5.3): no commits or releases within the 18 months preceding the May 2026 measurement date, and no current maintenance.

Table D.2: Packages excluded on activity grounds.

Package	**Reason for exclusion**
MRDS (Microsoft Robotics Developer Studio)	No updates since 2012; no longer supported by Microsoft.
USARSim (Unified System for Automation and Robot Simulation)	No active maintenance; does not represent current practice.

Excluded to Maintain Methodological Coherence

The packages in Table D.3 are public reference implementations of motion planning algorithms rather than general-purpose simulators, planning frameworks, physics engines, or middleware. Including them alongside Gazebo, MoveIt!, or ROS 2 would force a comparison between full software stacks and single-algorithm libraries and would distort both the commonality analysis and the cross-package measurements. They were excluded to maintain methodological consistency, not because of any quality concern.

Table D.3: Packages excluded to maintain methodological coherence.

Package	**Reason for exclusion**
GPMP2 (Gaussian Process Motion Planner 2)	Single-algorithm optimization-based motion planning reference implementation; narrower in scope than the general-purpose packages that dominate the candidate list.
CHOMP (Covariant Hamiltonian Optimization for Motion Planning)	Single-algorithm optimization-based motion planning reference implementation; narrower in scope than the general-purpose packages that dominate the candidate list.

The full filtered candidate list, including packages excluded at each step, is preserved in the supplementary project materials that accompany this report; Appendix A records the leading candidates with their mention counts and dispositions.

Appendix E

Interview Materials

Appendix F

Exploratory AI-Assisted Measurement

This appendix records a small exploratory trial in which an AI coding agent was asked to carry out the measurement protocol of this study on its own. The trial covered two physics engines from the selected set, PyBullet and MuJoCo, and used the template described in Appendix C together with the procedure in Section 3.7. The trial is not part of the validated measurement results in Chapter 5; it is reported here as a feasibility check and as the basis for the future-work direction in Section 10.4. The agent worked in a Linux shell with file-system and network access in early June 2026, so its repository snapshots fall slightly later than the May 2026 snapshot used for the main study.

Task Given to the Agent

The agent was given a single natural-language prompt and no installation commands. It had to locate the official documentation, install each package, verify the installation, and complete the template without further guidance. The prompt is reproduced verbatim below.

> Read the CSV file in the current directory and use the first column ("Metric") as the measurement template. Measure two software packages, PyBullet and MuJoCo. The requirements were:
>
> - Only fill in the columns for these two packages; do not modify the other software columns. A new CSV file may be created.
> - Do not rely on user-provided installation commands; find the official documentation, GitHub README, or project website to determine the installation and verification steps.
> - Follow each metric in the CSV to complete installation, verification, observation, and data entry.
> - Take a screenshot at every step and save it to a screenshots folder. Screenshots must cover at least: reading the CSV, finding the official documentation, the installation steps, the verification steps, error messages or successful output, and the final CSV result.
> - Evidence may come only from official documentation, GitHub, terminal output, and commands actually executed.
> - Write "unclear" for uncertain values and "n/a" for values that do not apply.
> - Also write a `notes.md` file recording the selected packages, the official links used, the commands run at each step, the screenshot filenames, whether

installation succeeded, whether verification succeeded, which CSV cells were filled or changed, and any problems encountered with their recovery.

Complete Bullet/PyBullet first, then MuJoCo.

Official Sources Located by the Agent

Working only from the prompt, the agent found the official sources in Table F.1 and used them to choose the installation and verification steps.

Table F.1: Official sources the agent used for the two packages.

Package	**Resource**	**Location**
PyBullet	Website and forum	`https://pybullet.org`
	Source repository	`https://github.com/bulletphysics/bullet3`
	Python package	`https://pypi.org/project/pybullet/`
MuJoCo	Website	`https://mujoco.org`
	Source repository	`https://github.com/google-deepmind/mujoco`
	Documentation	`https://mujoco.readthedocs.io`

Installation and Verification

Both packages installed with a single `pip` command inside a Python virtual environment, and both were verified with a short program that the agent wrote and ran.

Table F.2 summarizes the outcome.

Table F.2: Installation and verification outcome of the AI-assisted trial.

Item	**PyBullet**	**MuJoCo**
Install command	`pip install pybullet`	`pip install mujoco`
Installed version	3.2.7	3.9.0
Installation steps	1 (single pip command)	1 (single pip command)
Extra packages	none (self-contained)	bundled with the wheel
Operating system	Linux (Ubuntu)	Linux (Ubuntu)
Verification	Loaded the bundled `plane.urdf` model in DIRECT mode	Compiled an inline XML model of a free-jointed box on a plane
Result	Succeeded	Succeeded

Alongside the installation, the agent queried the GitHub API and recorded the project metadata that the template asks for. For PyBullet (`bulletphysics/bullet3`) it recorded 311 contributors, a zlib license, a first commit in April 2011, and a most recent push in October 2025. For MuJoCo (`google-deepmind/mujoco`) it recorded 116 contributors, an Apache-2.0 license, a first commit in August 2021, and a most recent push in June 2026. Each entry was tied to the source the agent read, either the official site, the repository README, or the GitHub API. The contributor counts come from the GitHub contributors API, which returns fewer entries than the commit-log method used for the main study (Section 3.7.2); the main-study records list 333 developers for Bullet/PyBullet and 138 for MuJoCo, so the two sets of numbers are not directly comparable.

Problems Encountered and Recovery

The agent encountered three problems and recovered from each without manual intervention. First, a direct `pip install` failed because the system Python was externally managed; the agent created a virtual environment and installed into it. Second, its first PyBullet verification script queried the physics engine before connecting to it; the agent reordered the calls so that the connection came first. Third, no screenshot utility was present, so the agent installed one before continuing.

Limitations and Relationship to the Main Study

This was a single run over two packages by one agent, so it shows feasibility rather than a result that belongs with the validated measurements in Chapter 5. The checkable, factual entries, such as the installation command, the installed version, the license, the contributor count, and the verification result, were each tied to a public source recorded by the agent and were straightforward to confirm. The subjective impression scores and the free-text judgements that the template also asks for were not validated here; in places they disagree with the agent's own notes, which is the clearest reason that a human still has to review the output before it can serve as evidence.

The artifacts from the trial, namely the completed template and the notes document, are preserved in the supplementary project materials that accompany this report. The trial motivates the future-work direction in Section 10.4, where agent-assisted measurement under human supervision is discussed as a way to scale State-of-the-Practice studies.

Bibliography

Richard J. Acton. Research software sharing, publication, & distribution checklists. Webinar, Best Practices for HPC Software Developers (HPC-BP) series, IDEAS Productivity, May 2026.

Davide Brugali, Arvin Agah, Bruce MacDonald, Issa A. D. Nesnas, and William D. Smart. Trends in robot software domain engineering. In *Software Engineering for Experimental Robotics*, volume 30 of *Springer Tracts in Advanced Robotics*. Springer, Berlin, Heidelberg, 2007. doi: 10.1007/978-3-540-68951-5_1.

Howie Choset, Kevin M. Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia E. Kavraki, and Sebastian Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, Cambridge, MA, 2005.

Jack Collins, Shelvin Chand, Anthony Vanderkop, and David Howard. A review of physics simulators for robotic applications. *IEEE Access*, 9:51416–51431, 2021. doi: 10.1109/ACCESS.2021.3068769.

Andrew Farley, Jie Wang, and Joshua A. Marshall. How to pick a mobile robot simulator: A quantitative comparison of CoppeliaSim, Gazebo, MORSE and Webots

with a focus on accuracy of motion. *Simulation Modelling Practice and Theory*, 117:102629, 2022. doi: 10.1016/j.simpat.2022.102629.

Jo Erskine Hannay, Carolyn MacLeod, Janice Singer, Hans Petter Langtangen, Dietmar Pfahl, and Greg Wilson. How do scientists develop and use scientific software? In *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pages 1–8, 2009. doi: 10.1109/SECSE.2009.5069155.

Adam Harris and James M. Conrad. Survey of popular robotics simulators, frameworks, and toolkits. In *Proceedings of IEEE SoutheastCon 2011*, pages 243–249, Nashville, TN, USA, 2011. doi: 10.1109/SECON.2011.5752942.

Pablo Iñigo-Blasco, Fernando Diaz-del Rio, Ma Carmen Romero-Ternero, Daniel Cagigas-Muñiz, and Saturnino Vicente-Diaz. Robotics software frameworks for multi-agent robotic systems development. *Robotics and Autonomous Systems*, 60 (6):803–821, 2012. doi: 10.1016/j.robot.2012.02.004.

Seyed Mohamad Kargar, Borislav Yordanov, Carlo Harvey, and Ali Asadipour. Emerging trends in realistic robotic simulations: A comprehensive systematic literature review. *IEEE Access*, 12, 2024.

Marian Körber, Johann Lange, Stephan Rediske, Simon Steinmann, and Roland Glück. Comparing popular simulation environments in the scope of robotics and reinforcement learning, 2021.

Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, New York, NY, USA, 2006.

Peter Michalski. State of the practice for lattice boltzmann method software. M.eng. thesis, McMaster University, 2021.

David Lorge Parnas. On the design and development of program families. *IEEE Transactions on Software Engineering*, SE-2(1):1–9, 1976.

Prakash Prabhu, Thomas B. Jablin, Arun Raman, Yun Zhang, Jialu Huang, Hanjun Kim, Nick P. Johnson, Feng Liu, Soumyadeep Ghosh, Stephen Beard, Taewook Oh, Matthew Zoufaly, David Walker, and David I. August. A survey of the practice of computational science. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12, New York, NY, USA, 2011. ACM. doi: 10.1145/2063348.2063374.

Michael Reckhaus, Nico Hochgeschwender, Jan Paulus, Azamat Shakhimardanov, and Gerhard K. Kraetzschmar. An overview about simulation and emulation in robotics. In *Proceedings of the Workshop on Simulation Technologies in the Robot Development Process, International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR 2010)*, Darmstadt, Germany, 2010.

Thomas L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. McGraw-Hill Publishing Company, New York, New York, 1980.

W. Spencer Smith, D. Adam Lazzarato, and Jacques Carette. State of the practice for mesh generation and mesh processing software. *Advances in Engineering Software*, 100:53–71, 2016.

W. Spencer Smith, D. Adam Lazzarato, and Jacques Carette. State of the practice for GIS software, 2018a.

W. Spencer Smith, Yue Sun, and Jacques Carette. Statistical software for psychology: Comparing development practices between CRAN and other communities, 2018b.

W. Spencer Smith, Zheng Zeng, and Jacques Carette. Seismology software: State of the practice. *Journal of Seismology*, 22:755–788, 2018c. doi: 10.1007/s10950-018-9731-3.

W. Spencer Smith, Jacques Carette, Oluwaseun Owojaiye, Peter Michalski, and Ao Dong. Methodology for assessing the state of the practice for domain X, 2021.

W. Spencer Smith, Ao Dong, Jacques Carette, and Michael D. Noseworthy. State of the practice for medical imaging software, 2024a.

W. Spencer Smith, Peter Michalski, Jacques Carette, and Zahra Keshavarz-Motamed. State of the practice for lattice boltzmann method software. *Archives of Computational Methods in Engineering*, 31:313–350, 2024b. doi: 10.1007/s11831-023-09981-2.

Aaron Staranowicz and Gian Luca Mariottini. A survey and comparison of commercial and open-source robotic simulator software. In *Proceedings of the 4th International Conference on PErvasive Technologies Related to Assistive Environments (PETRA '11)*, pages 1–8. ACM, 2011. doi: 10.1145/2141622.2141689.

David M. Weiss. Commonality analysis: A systematic process for defining families. In *Development and Evolution of Software Architectures for Product Families (ARES '98)*, volume 1429 of *Lecture Notes in Computer Science*, pages 214–222. Springer, Berlin, Heidelberg, 1998. doi: 10.1007/3-540-68383-6_30.

Leon Žlajpah. Simulation in robotics. *Mathematics and Computers in Simulation*, 79 (4):879–897, 2008.